รายงานโครงงาน **Intro ML**

บทที่ **1** บทนำ

ฐานข้อมูลลูกค้าชุดเดียวใช้ตอบการตัดสินใจสองข้อ คือ ควรเสนอแคมเปญให้ลูกค้ารายใด (Response) และ ลูกค้ารายนั้นมีรายได้ต่อปีเท่าใด (Income) โครงงานนี้สร้างและเปรียบเทียบ Model ตามลำดับเวลาของการพัฒนา ตั้งแต่โมเดลพื้นฐานจนถึงโมเดลนอกเหนือจากที่เรียนในห้อง

1.1 ที่มาและความสำคัญ

- บริษัทขายปลีกมีลูกค้าจำนวนมาก แต่มีเพียง 14.9% ที่ตอบรับแคมเปญล่าสุด การจัดอันดับลูกค้าที่มีโอกาสตอบรับสูงช่วยลดต้นทุนการส่งข้อเสนอ
- Income เป็นข้อมูลอ่อนไหวและมักขาดหาย (ในชุดข้อมูลนี้ขาด 24 แถว) จึงต้องประมาณจากพฤติกรรมการซื้อและช่องทางที่ลูกค้าใช้
- ชุดข้อมูลเดิมสร้างมาเพื่อแบ่งกลุ่มลูกค้า (clustering) เรานำมาใช้ใหม่เพื่อการพยากรณ์แบบมีผู้สอน (supervised) สองงาน

1.2 วัตถุประสงค์

1. สร้างโมเดล classification Response
2. สร้างโมเดล regression Income
3. ทุกโมเดลมีสองแบบ คือ Library และ From Scratch
4. สร้างโมเดลใหม่นอกเหนือจากที่เรียนอย่างน้อยงานละหนึ่งโมเดล และอธิบายกลไกภายใน
5. ตรวจสอบความทนทานของผลด้วย 5-fold cross-validation

1.3 ชุดข้อมูล

รายการ	รายละเอียด
แหล่งที่มา	Kaggle `imakash3011/customer-personality-analysis` (CC0)
ขนาด	2,240 แถว × 29 คอลัมน์ คั่นด้วย tab
กลุ่มคอลัมน์	People, Products, Promotion, Place
classification target	`Response` (14.94% ตอบรับ / 85.06% ไม่ตอบรับ)
regression target	`Income` (เฉลี่ย $51,959 มัธยฐาน $51,371)
cleaned	2,236 แถว (clf) / 2,212 แถว (reg)

1.4 ขอบเขตและตัวชี้วัด

classify ใช้ **F1** เป็น Main Metric เพราะการเดา "ไม่ตอบรับ" ทุกแถวได้ accuracy 0.851 อยู่แล้ว regression ใช้ **R^2** เป็น Main Metric และรายงาน RMSE (หน่วยดอลลาร์) เป็น Sub Metrics

$$F1 = \frac{2 \cdot P \cdot R}{P + R}, \qquad P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

ตัวชี้วัดอื่นที่รายงานเพิ่ม: BCE loss, accuracy, precision, recall (sensitivity), ROC-AUC สำหรับ classify และ MSE, RMSE, MAE สำหรับ regression

1.5 โครงสร้างรายงาน

- บทที่ 2 การวิเคราะห์ข้อมูลเชิงสำรวจ (EDA) และการคัดเลือกฟีเจอร์
- บทที่ 3 ระเบียบวิธีและการตั้งค่าการทดลอง
- บทที่ 4 ลำดับการพัฒนาโมเดลและเหตุผลเบื้องหลัง (V1.0 ถึง V3.1)
- บทที่ 5 ผลการทดลองเปรียบเทียบและการอภิปราย
- บทที่ 6 สรุปและข้อเสนอแนะ

บทที่ **2** การวิเคราะห์ข้อมูลเชิงสำรวจ **(EDA)**

จาก 29 คอลัมน์เหลือ 25 ตัวแปร ใช้การโหวต 3 วิธีเลือกฟีเจอร์เหลือ **18** ตัว **(22** หลัง **one-hot)** สำหรับ **Response** และ **13** ตัว สำหรับ **Income** บทนี้สรุปผล EDA

2.1 ความหมายของ **Feature**

กลุ่ม	Feature	ความหมาย
People	ID	รหัสประจำตัวลูกค้า
People	Year_Birth	ปีเกิดของลูกค้า
People	Education	ระดับการศึกษา
People	Marital_Status	สถานะสมรส
People	Income	รายได้ต่อปีของครัวเรือน (ดอลลาร์)
People	Kidhome	จำนวนเด็กเล็กในบ้าน
People	Teenhome	จำนวนวัยรุ่นในบ้าน
People	Dt_Customer	วันที่สมัครเป็นลูกค้า
People	Recency	จำนวนวันนับจากการซื้อครั้งล่าสุด
People	Complain	1 = เคยร้องเรียนใน 2 ปีที่ผ่านมา, 0 = ไม่เคย
Products	MntWines	ยอดซื้อไวน์ใน 2 ปีที่ผ่านมา
Products	MntFruits	ยอดซื้อผลไม้ใน 2 ปีที่ผ่านมา
Products	MntMeatProducts	ยอดซื้อเนื้อสัตว์ใน 2 ปีที่ผ่านมา
Products	MntFishProducts	ยอดซื้อปลาใน 2 ปีที่ผ่านมา
Products	MntSweetProducts	ยอดซื้อขนมหวานใน 2 ปีที่ผ่านมา
Products	MntGoldProds	ยอดซื้อสินค้าทองคำใน 2 ปีที่ผ่านมา
Promotion	NumDealsPurchases	จำนวนครั้งที่ซื้อด้วยส่วนลด

กลุ่ม	Feature	ความหมาย
Promotion	AcceptedCmp1	1 = ตอบรับข้อเสนอแคมเปญที่ 1, 0 = ไม่ตอบรับ
Promotion	AcceptedCmp2	1 = ตอบรับข้อเสนอแคมเปญที่ 2, 0 = ไม่ตอบรับ
Promotion	AcceptedCmp3	1 = ตอบรับข้อเสนอแคมเปญที่ 3, 0 = ไม่ตอบรับ
Promotion	AcceptedCmp4	1 = ตอบรับข้อเสนอแคมเปญที่ 4, 0 = ไม่ตอบรับ
Promotion	AcceptedCmp5	1 = ตอบรับข้อเสนอแคมเปญที่ 5, 0 = ไม่ตอบรับ
Promotion	Response	1 = ตอบรับข้อเสนอแคมเปญล่าสุด, 0 = ไม่ตอบรับ (เป้าหมาย classify)
Place	NumWebPurchases	จำนวนครั้งที่ซื้อผ่านเว็บไซต์
Place	NumCatalogPurchases	จำนวนครั้งที่ซื้อผ่านแคตตาล็อก
Place	NumStorePurchases	จำนวนครั้งที่ซื้อที่หน้าร้าน
Place	NumWebVisitsMonth	จำนวนครั้งที่เข้าเว็บไซต์ในเดือนที่ผ่านมา
ค่าคงที่	Z_CostContact	เท่ากับ 3 ทุกแถว ไม่ระบุความหมาย (ลบทิ้ง)
ค่าคงที่	Z_Revenue	เท่ากับ 11 ทุกแถว ไม่ระบุความหมาย (ลบทิ้ง)
สร้างเพิ่ม	Age	อายุ = 2014 – Year_Birth
สร้างเพิ่ม	Customer_Tenure_Days	จำนวนวันจากวันสมัครถึงวันสมัครล่าสุดในข้อมูล (2014-06-29)

2.2 การ **Clean Data**

ทำการ clean data ตาม 7 ขั้นตอนดังต่อไปนี้

ลำดับ	การดำเนินการ	เหตุผล / ผล
1	ลบ ID	เป็นรหัสประจำตัวไม่ใช่ฟีเจอร์
2	ลบ Z_CostContact, Z_Revenue	ค่าคงที่ ไม่มีความแปรปรวน
3	แปลง Dt_Customer เป็น Customer_Tenure_Days	จำนวนวันจากวันสมัครถึงวันสมัครล่าสุด (2014-06-29) ช่วง 0–699 วัน เฉลี่ย 353.6 วัน
4	สร้าง Age = 2014 – Year_Birth และลบ 3 แถวที่ปีเกิดผิดปกติ (1893, 1899, 1900)	อายุ 121, 115, 114 ปี เป็นข้อผิดพลาดการกรอก แถวเหลือ 2,237
5	รวม Absurd, YOLO, Alone เข้ากับ Single	สองค่าแรกเป็นคำตอบล้อเล่น Alone ความหมายเท่า Single และมีจำนวนน้อย Single เพิ่มจาก 479 เป็น 486 เหลือ 5 หมวด
6	Income หาย 24 แถว	classify: เก็บแถวไว้และเติมค่ามัธยฐานในฐานะฟีเจอร์ regression: ลบ 24 แถวนั้น เพราะ Income คือเป้าหมาย
7	ลบแถวลูกค้า 9432 ที่ Income = $666,666	ค่าผิดปกติสุดขีด รายได้สูงสุดหลังลบเหลือ $162,397

2.3 การกระจายตัวของตัวแปรเป้าหมายและการแบ่งชุดข้อมูล

แบ่งชุดข้อมูล 80/20 ด้วย `random_state = 42` (stratified)

ชุด	n	Yes (1)	No (0)
Full (classify)	2,236	334 (14.94%)	1,902 (85.06%)
Train	1,788	267 (14.93%)	1,521 (85.07%)
Test	448	67 (14.96%)	381 (85.04%)

Income	Full (2,212)	Test (n = 443)
ค่าเฉลี่ย	$51,959	$53,038
มัธยฐาน	$51,371	$51,948
ส่วนเบี่ยงเบนมาตรฐาน	$21,527	$21,819
ต่ำสุด / สูงสุด	$1,730 / $162,397	$4,023 / $162,397

2.4 ความสัมพันธ์กับตัวแปรเป้าหมาย **(relevance)**

ใช้สองตัววัดตามชนิดฟีเจอร์: ตัวเลขใช้ point-biserial (classify) หรือ Pearson (regression) หมวดใช้ Cramér's V หรืออัตราส่วนสหสัมพันธ์ η เกณฑ์ผ่านคือ 0.10 ทั้ง `Response` และ `Income` มีสัมพันธ์ชัดกว่าชุดข้อมูลอื่นที่เคยทำมา

classify `Response`: 19 จาก **25** ตัวผ่าน **0.10**

- สูงสุดคือ `AcceptedCmp5` (0.328), `AcceptedCmp1` (0.294), `AcceptedCmp3` (0.254), `MntWines` (0.247) และ `MntMeatProducts` (0.237)
- `AcceptedCmp1–5` ไม่ใช่ **data leakage**: เป็นประวัติคามเปญในอดีต มีอยู่ก่อนแคมเปญของ `Response` เสมอ อัตราตอบรับเท่ากับ **40.7%** (n = 462) หากเคย

ตอบรับแคมเปญใดแคมเปญหนึ่ง เทียบกับ **8.2%** (n = 1,775) หากไม่เคย ข้อจำกัดคือลูกค้าใหม่ที่ไม่มีประวัติจะไม่มีฟีเจอร์กลุ่มนี้ (cold-start)

- `Recency` (0.199) และ `Customer_Tenure_Days` (0.194) สอดคล้องกับหลัก RFM: ซื้อล่าสุดใกล้กว่าและอยู่กับบริษัทนานกว่า ตอบรับมากกว่า
- `Teenhome` (0.155) ผ่าน แต่ `Kidhome` (0.080) ไม่ผ่าน บ้านที่มี `Teenhome` ตอบรับมากกว่าบ้านที่มี `Kidhome`
- ไม่ผ่าน: `Kidhome`, `NumStorePurchases` (0.039), `Age` (0.018), `NumWebVisitsMonth` (0.004), `NumDealsPurchases` (0.002), `Complain` (0.0002)

regression `Income`: 17 จาก **25** ตัวผ่าน **0.10** ซึ่งแข็งแกร่งกว่า **classify** มาก

- สูงสุดคือ `NumCatalogPurchases` (0.589), `MntMeatProducts` (0.584), `MntWines` (0.578) และ `NumWebVisitsMonth` (−0.553)
- `NumWebVisitsMonth` เป็นความสัมพันธ์เชิงลบ ผู้ที่มี `NumWebVisitsMonth` สูง มีรายได้น้อยกว่า และ `Kidhome` (−0.428) บ้านที่มี `Kidhome` สูงมีรายได้เฉลี่ยต่ำกว่า
- `Education` (η = 0.218): ระดับ `Basic` มีรายได้เฉลี่ย $20,306 ต่ำกว่าระดับอื่นประมาณ 2.5 เท่า (ระดับอื่นอยู่ในช่วง $47.6k–$56.1k)
- ผู้ตอบรับแคมเปญมีรายได้มัธยฐาน $64,090 สูงกว่าผู้ไม่ตอบรับ ($50,150) สูงกว่า **27.8%** ซึ่งเป็นความสัมพันธ์เดียวกันกับ `Income` ใน classify (0.133) มองจากอีกด้าน

2.5 ความสัมพันธ์ร่วมระหว่างฟีเจอร์ **(multicollinearity)**

คำนวณที่ Pearson ระหว่างฟีเจอร์เชิงตัวเลข 23 ตัว (ฟีเจอร์กลุ่มใหญ่ที่เกี่ยวกันแรงกว่าชุดข้อมูลอื่น) คู่ที่สัมพันธ์สูงสุดคือ

คู่ฟีเจอร์	r
MntMeatProducts – NumCatalogPurchases	0.724
MntWines – NumStorePurchases	0.642
MntWines – NumCatalogPurchases	0.635
MntFruits – MntFishProducts	0.594
Income – NumCatalogPurchases	0.589
Income – NumWebVisitsMonth	–0.553

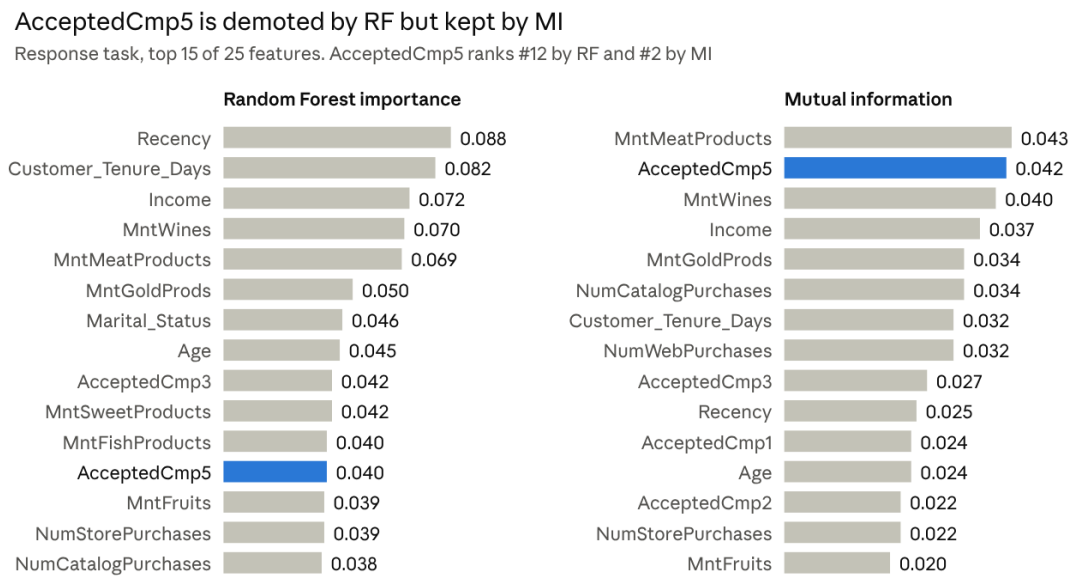
ฟีเจอร์ Income, MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds และ NumWebPurchases, NumCatalogPurchases, NumStorePurchases เคลื่อนไหวไปด้วยกัน เป็นภาพลักษณ์ของ "ลูกค้ามั่งคั่งที่ซื้อมากผ่านแคตตาล็อกและหน้าร้าน" กลุ่มเดียว ไม่ใช่หลายมิติ อิสระ Education กับ Marital_Status ไม่ซ้ำซ้อนกัน (Cramér's V = 0.043)

2.6 ความสำคัญของฟีเจอร์จากโมเดล (RF) และ Mutual Information

ตัววัด bivariate ดูทีละฟีเจอร์ จึงมองไม่เห็นปฏิสัมพันธ์และความสัมพันธ์ไม่เชิงเส้น เราจึงเพิ่มอีกสองมุมมองที่วัดสิ่งต่างกัน

- **Random Forest:** n_estimators = 300, random_state = 42 ความสำคัญรวมเท่ากับ 1 และ dummy ของ Education / Marital_Status ถูกรวมกลับเป็นคอลัมน์แม่
- **Mutual Information (MI):** mutual_info_classif / mutual_info_regression ฟีเจอร์ตัวเลขใช้ discrete_features=False ฟีเจอร์หมวดหมู่ใช้ True

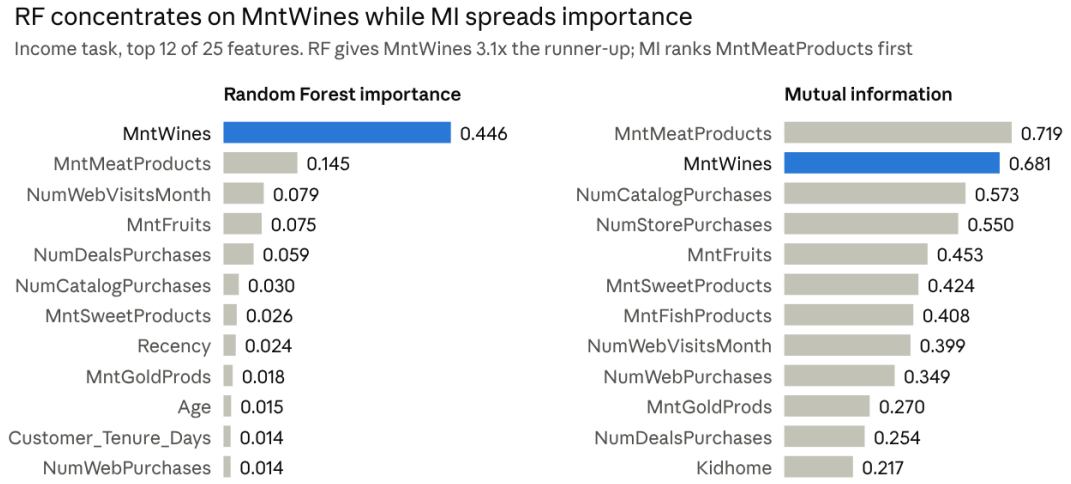
classify Response (แสดง top 15 จาก 25)



classify.ipynb · RF 300 trees, seed 42; MI k-NN estimator · 2,237 rows

- RF ดัน Recency (0.088) และ Customer_Tenure_Days (0.082) ขึ้นอันดับ 1–2 ทั้งที่ bivariate ให้เพียงอันดับ 7–8
- AcceptedCmp5 ที่เป็นอันดับ 1 ของ bivariate ตกเป็นอันดับ 12 ใน RF (0.040) แต่ MI ยังให้อันดับ 2 (0.042)
- ทั้งสามวิธีจึงเห็นไม่ตรงกันว่า AcceptedCmp1, AcceptedCmp2, AcceptedCmp3, AcceptedCmp4, AcceptedCmp5 ควรมีน้ำหนักเท่าใดเมื่อเทียบกับ Recency และ MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds ไม่มีวิธีใดเป็นคำตอบสุดท้าย
- การที่ RF ลดอันดับ AcceptedCmp1, AcceptedCmp2, AcceptedCmp3, AcceptedCmp4, AcceptedCmp5 เป็นผลของความซ้ำซ้อนระหว่างฟีเจอร์ในกลุ่มเดียวกัน ไม่ใช่หลักฐานว่าฟีเจอร์เหล่านี้ผิดปกติ

regression Income (แสดง top 12 จาก 25)



regression.ipynb · RF 300 trees, seed 42; MI k-NN estimator · 2,213 rows

- RF ให้ MntWines 0.446 ซึ่งมากกว่าอันดับสอง MntMeatProducts (0.145) ถึง 3 เท่า
- NumCatalogPurchases ที่เป็นอันดับ 1 ของ bivariate ตกเป็นอันดับ 6 (0.030) เพราะมันสัมพันธ์กับ MntWines และ MntMeatProducts (r = 0.57–0.73) เมื่อ RF เลือกสองตัวนั้นแยกก่อน ข้อมูลที่เหลือให้ตัวอื่นก็น้อยลง
- MI เรียงลำดับคล้าย bivariate: MntMeatProducts (0.719) นำ MntWines (0.681) และ NumCatalogPurchases / NumStorePurchases ยังสูงทั้งที่ RF ลดอันดับ ยืนยันว่าเป็นผลของ RF ไม่ใช่ฟีเจอร์อ่อน
- ค่า MI สูงสุดของ regression (0.719) มากกว่า classify (0.043) ประมาณ 17 เท่า สอดคล้องกับข้อสังเกตว่า Income มีสัญญาณที่เรียนรู้ได้ชัดกว่า Response มาก
- Response ซึ่งผ่านเกณฑ์ bivariate (0.133) ได้ RF เพียง 0.0005 (อันดับ 22) และ MI 0.038 (อันดับ 21) แปลว่าเมื่อมีฟีเจอร์ MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds และ NumWebPurchases, NumCatalogPurchases, NumStorePurchases อยู่แล้ว Response แทบไม่เพิ่มข้อมูลใหม่

2.7 การโหวต 3 วิธีเพื่อเลือกฟีเจอร์ (three-vote rule)

คะแนนสามวิธีอยู่คละสเกล (bivariate เป็นค่ามีเครื่องหมาย –1 ถึง +1, RF รวมเท่ากับ 1, MI ไม่มีเพดาน) การเฉลี่ยจึงทำให้วิธีที่มีช่วงค่ากว้างที่สุดครองผล เราจึงให้แต่ละวิธีลงคะแนน “ผ่าน / ไม่ผ่าน” หนึ่งเสียงต่อฟีเจอร์

วิธี	ผ่านเมื่อ	classify	regression
Bivariate	ค่าสัมบูรณ์ ≥ 0.10	19 จาก 25	17 จาก 25
Random Forest	อยู่ใน top-N	N = 15	N = 12
Mutual Information	อยู่ใน top-N	N = 15	N = 12

จำนวนโหวตแบ่งฟีเจอร์เป็น 4 ชั้น

ชั้น	ความหมาย	จำนวน classify	จำนวน regression
3/3	ทุกวิธีเห็นตรงกัน เลือก	10	8
2/3	สองในสามเห็นตรงกัน พิจารณารายตัว	8	5
1/3	วิธีเดียวผ่าน ตัด	3	7
0/3	ไม่ผ่านเลย ตัด	4	5

2.8 ผลการโหวตชั้น 3/3 และ 2/3

classify Response — ชั้น 3/3 (10 ตัว)

ฟีเจอร์	Bivariate	RF (อันดับ)	MI (อันดับ)
AcceptedCmp5	0.328 ✓	0.040 ✓ (12)	0.042 ✓ (2)
AcceptedCmp3	0.254 ✓	0.042 ✓ (9)	0.027 ✓ (9)
MntWines	0.247 ✓	0.070 ✓ (4)	0.040 ✓ (3)
MntMeatProducts	0.237 ✓	0.069 ✓ (5)	0.043 ✓ (1)

ฟีเจอร์	Bivariate	RF (อันดับ)	MI (อันดับ)
NumCatalogPurchases	0.221 ✓	0.038 ✓ (15)	0.034 ✓ (6)
Recency	0.199 ✓	0.088 ✓ (1)	0.025 ✓ (10)
Customer_Tenure_Days	0.194 ✓	0.082 ✓ (2)	0.032 ✓ (7)
MntGoldProds	0.141 ✓	0.050 ✓ (6)	0.034 ✓ (5)
Income	0.133 ✓	0.072 ✓ (3)	0.037 ✓ (4)
MntFruits	0.126 ✓	0.039 ✓ (13)	0.020 ✓ (15)

classify Response — ชั้น 2/3 (8 ตัว) ช่องที่ไม่ผ่านเป็น ✗

ฟีเจอร์	Bivariate	RF (อันดับ)	MI (อันดับ)
AcceptedCmp1	0.294 ✓	0.033 × (17)	0.024 ✓ (11)
AcceptedCmp2	0.169 ✓	0.008 × (23)	0.022 ✓ (13)
Marital_Status	0.152 ✓	0.046 ✓ (7)	0.011 × (20)
NumWebPurchases	0.148 ✓	0.030 × (19)	0.032 ✓ (8)
MntSweetProducts	0.117 ✓	0.042 ✓ (10)	0.017 × (17)
MntFishProducts	0.111 ✓	0.040 ✓ (11)	0.006 × (21)
NumStorePurchases	0.039 ×	0.039 ✓ (14)	0.022 ✓ (14)
Age	0.018 ×	0.045 ✓ (8)	0.024 ✓ (12)

ชั้น 1/3 (3 ตัว): AcceptedCmp4, Teenhome, Education · ชั้น 0/3 (4 ตัว): Kidhome, NumWebVisitsMonth, NumDealsPurchases, Complain

regression Income — ชั้น 3/3 (8 ตัว)

ฟีเจอร์	Bivariate	RF (อันดับ)	MI (อันดับ)
NumCatalogPurchases	0.589 ✓	0.030 ✓ (6)	0.573 ✓ (3)
MntMeatProducts	0.584 ✓	0.145 ✓ (2)	0.719 ✓ (1)
MntWines	0.578 ✓	0.446 ✓ (1)	0.681 ✓ (2)
NumWebVisitsMonth	−0.553 ✓	0.079 ✓ (3)	0.399 ✓ (8)
MntSweetProducts	0.441 ✓	0.026 ✓ (7)	0.424 ✓ (6)
MntFruits	0.430 ✓	0.075 ✓ (4)	0.453 ✓ (5)
NumWebPurchases	0.388 ✓	0.014 ✓ (12)	0.349 ✓ (9)
MntGoldProds	0.325 ✓	0.018 ✓ (9)	0.270 ✓ (10)

regression Income — ชั้น 2/3 (5 ตัว) ช่องที่ไม่ผ่านเป็น ✗

ฟีเจอร์	Bivariate	RF (อันดับ)	MI (อันดับ)
NumStorePurchases	0.530 ✓	0.008 × (15)	0.550 ✓ (4)
MntFishProducts	0.439 ✓	0.012 × (13)	0.408 ✓ (7)
Kidhome	−0.428 ✓	0.004 × (19)	0.217 ✓ (12)
Age	0.163 ✓	0.015 ✓ (10)	0.130 × (14)
NumDealsPurchases	−0.083 ×	0.059 ✓ (5)	0.254 ✓ (11)

ชั้น 1/3 (7 ตัว): AcceptedCmp5, AcceptedCmp1, Education, AcceptedCmp4, Response, Customer_Tenure_Days, Recency · ชั้น 0/3 (5 ตัว): AcceptedCmp2, Marital_Status, Complain, Teenhome, AcceptedCmp3

สิ่งที่ตารางแสดง:

- ชั้น 2/3 ที่ตกเพียง bivariate (NumStorePurchases, Age ใน classify · NumDealsPurchases ใน regression) ผ่านทั้ง RF และ MI จึงมีสัญญาณไม่เชิงเส้นที่ค่าสหสัมพันธ์เชิงเส้นมองไม่เห็น

- ชั้น 2/3 ที่ตกเพียง RF (`NumStorePurchases`, `MntFishProducts`, `Kidhome` ใน regression) เป็นผลของ greedy splitting ในกลุ่มฟีเจอร์ที่สัมพันธ์กัน (หัวข้อ 2.6)
- ชั้น 2/3 ที่ตกเพียง MI (`MntSweetProducts`, `MntFishProducts` ใน classify) อาจเกิดจากตัวประมาณ MI ของเป้าหมายทวินารีซึ่งมีความแปรปรวนสูงกว่า ผลพลาดชิดเกณฑ์จึงไม่เป็นหลักฐานที่แข็งแกร่ง
- `AcceptedCmp1` / `AcceptedCmp2` ตกที่ RF ทั้งคู่ สอดคล้องกับการที่ RF ลดอันดับทั้งกลุ่ม `AcceptedCmp1`, `AcceptedCmp2`, `AcceptedCmp3`, `AcceptedCmp4`, `AcceptedCmp5`

บทที่ **3** ระเบียบวิธีและการตั้งค่าการทดลอง

ทุกโมเดลในบทที่ 4 ใช้ชุดข้อมูล การแบ่ง และตัววัดชุดเดียวกัน เพื่อให้สามารถเปรียบเทียบระหว่างเวอร์ชัน บทนี้สรุปกติกาที่ใช้ร่วมกัน รวมถึงข้อจำกัดของวิธีประเมิน

3.1 สิ่งที่ใช้ร่วมกัน

รายการ	classify (Response)	regression (Income)
จำนวนแถว	2,236	2,212
การแบ่งชุดข้อมูล	80/20, `random_state = 42`, stratified	80/20, `random_state = 42`
จำนวนแถวชุดทดสอบ	448 (บวก 67)	443
Input Shape	22	13
Main Metric	**F1**	**R^2**
Sub Metrics	ROC-AUC, precision, recall, accuracy, BCE	RMSE, MAE, MSE

3.2 การเตรียม **Data**

- `StandardScaler` fit เฉพาะชุด train แล้วใช้กับชุด test เพื่อไม่ให้ข้อมูลชุด test รั่วเข้าการเทรน

- classify: `Income` ที่หาย 24 แถวถูกเติมด้วยมัธยฐานของชุด train (51,651) ในฐานะฟีเจอร์
- `Marital_Status` ทำ one-hot 5 มิติ ทำให้ classify มี 22 มิติ regression ไม่ใช้ฟีเจอร์หมวดหมู่จึงคง 13 มิติ
- ไม่ทำ data augmentation
- classify ใช้ weighted BCE ด้วย `pos_weight = neg / pos ≈ 5.7` เพื่อชดเชยความไม่สมดุล (ค่า BCE ที่รายงานคือ unweighted mean BCE)

3.3 แนวคิด **Library + Scratch**

ทุกโมเดลจะมี 2 version เสมอ

- **Library:** สร้างด้วย libraryใช้เป็นคำตอบอ้างอิง
- **Scratch twin:** เขียนเองด้วยทั้ง forward, backward และ optimizer จากสมการชุดเดียวกัน

ผลที่ได้จากโค้ดที่เขียนเองเชื่อถือได้เมื่อตรงกับ Library และความต่างที่เหลือมาจากการสุ่ม (ค่าเริ่มต้น ลำดับข้อมูล) ไม่ใช่ข้อผิดพลาดของสมการ

3.4 การตั้งค่าการฝึกแต่ละกลุ่มเวอร์ชัน

เวอร์ชัน	Optimizer	จำนวนรอบ	การควบคุมอื่น
V1.0	Adam (lr 1e-3, β = 0.9 / 0.999)	50 epochs, batch 64	ไม่มี schedule ไม่มี weight decay เก็บรอบสุดท้าย
V1.1, V1.2	Adam (lr 1e-3) + cosine annealing	100 epochs	weight decay 1e-4, BatchNorm, dropout, residual · เก็บรอบสุดท้าย
V2.0	Adam (lr 1e-3) + cosine (T_max = 100)	100 epochs	weight decay 1e-4 · เก็บ checkpoint ที่ F1 ดีสุด (classify) หรือ RMSE ต่ำสุด (regression)

เวอร์ชัน	Optimizer	จำนวนรอบ	การควบคุมอื่น
V3.0	Gradient boosting (ต้นไม้ CART)	จำนวนต้นไม้ให้ early stopping	ความลึกสูงสุด 5, min leaf 20 · checkpoint ที่ BCE ต่ำสุด
V1.3, V2.1, V3.1	เหมือน V1.1, V2.0, V3.0 ตามลำดับ	เหมือนเดิม	ใช้ค่าเดิมทั้งหมด เปลี่ยนแค่วิธีแบ่งข้อมูลเป็น 5-fold

3.5 ตัววัดผล

classify ใช้ F1 เป็นตัวหลัก เพราะคลาสบวกมีเพียง 14.9% F1 คือค่าเฉลี่ยฮาร์โมนิกของ precision และ recall ซึ่งคำนวณจาก TP, FP, FN เท่านั้น

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}$$

regression ใช้ R^2 เป็นตัวหลัก คือสัดส่วนความแปรปรวนของรายได้ที่โมเดลอธิบายได้ ส่วน RMSE แสดงเป็นหน่วยดอลลาร์เพื่อให้เห็นขนาดความคลาดเคลื่อน

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \qquad \text{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$$

3.6 การประเมินแบบ **k-fold** และข้อจำกัดของวิธีประเมิน

ชุด test มีเพียง 448 แถวใน classify (ลูกค้าที่ตอบรับ 67 แถว) และ 443 แถวใน regression ผลจาก split เดียวจึงสั่นไหวได้ง่าย จึงเพิ่มการตรวจ k-fold ในสามเวอร์ชันหลัง

- `k = 5`, `shuffle = True`, `random_state = 42` (classify - `StratifiedKFold`, regression - `KFold`) ทำบนชุดที่ clean แล้วทั้งชุด
- V1.3 ฝึกคงที่ 100 epochs ต่อ fold ไม่เลือก checkpoint เหมือน V1.1 บน split เดียว
- V2.1 และ V3.1 เลือก checkpoint ต่อ fold เหมือน V2.0 และ V3.0 ผลรายงานเป็นค่าเฉลี่ย ± ส่วนเบี่ยงเบนมาตรฐานของสองฝั่ง (Library และ scratch)

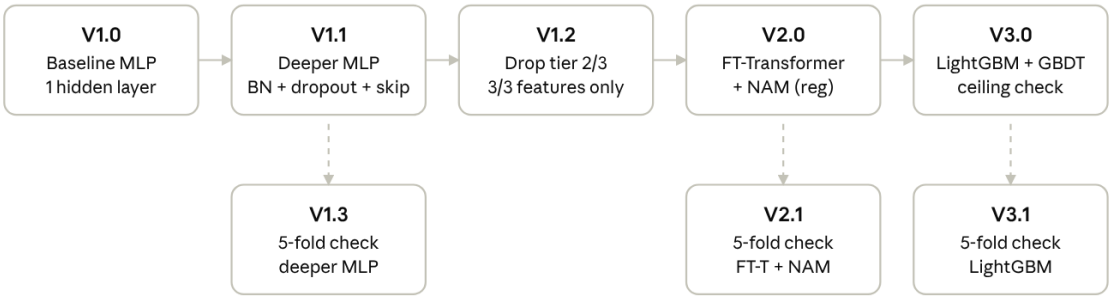
3.7 timeline การพัฒนา

ลำดับสร้าง	เวอร์ชัน	สิ่งที่เพิ่ม
1	V1.0	Baseline: MLP 1 hidden layer
2	V1.1	MLP หลาย Layer แบบ ResNet (BatchNorm, dropout, residual, cosine schedule)
3	V1.2	ตัดชั้น 2/3 เหลือเฉพาะ 3/3
4	V2.0	architecture ใหม่: FT-Transformer-lite (classify) และ NAM (regression)
5	V3.0	Gradient boosting (LightGBM + ประมวลผลเอง) เป็นตัวเทียบที่ดีที่สุด
6	V1.3	5-fold: MLP หลาย Layer
7	V2.1	5-fold: architecture ใหม่
8	V3.1	5-fold: gradient boosting

บทที่ **4** วิวัฒนาการของโมเดลตามลำดับเวอร์ชัน

บทนี้เล่าเหตุผลของทุกการตัดสินใจตามลำดับเวอร์ชัน แต่ละหัวข้อมี 5 ขั้นเสมอ

1. คำถาม ที่เวอร์ชันนี้ต้องตอบ
2. **architecture** และการตั้งค่าที่เปลี่ยนไป
3. ผลลัพธ์ บนชุดทดสอบชุดเดียวกัน (n = 448 classify · 443 regression) ทั้ง PyTorch (Library) และ Scratch (NumPy)
4. ทำไมจึงเป็นเช่นนั้น อธิบายผลด้วยหลักฐาน
5. คำถามถัดไป ที่ผลนั้นทิ้งไว้

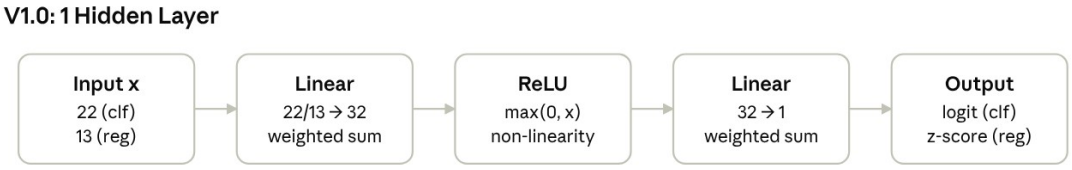


Version sequence

4.1 V1.0 — Baseline: MLP 1 Layer

1) คำถาม**:** โมเดลที่ง่ายที่สุดทำได้ดีแค่ไหน

2) architecture



architecture V1.0

- Initial Weight แบบ He
- Adam (lr 1e-3)
- 50 epochs
- batch 64
- seed 42
- regression ฝึกบน `Income` ที่ทำ z-score และคำนวณทุกตัวชี้วัดหลังแปลงกลับเป็นดอลลาร์

3) ผลลัพธ์ — **classify (`Response`)**

ตัววัด	PyTorch	Scratch	ส่วนต่าง
F1	**0.5161**	**0.4974**	**0.0187**
ROC-AUC	0.8669	0.8587	0.0082
Precision	0.4034	0.3852	0.0182
Recall	0.7164	0.7015	0.0149
Accuracy	0.7991	0.7879	0.0112
BCE	0.4028	0.4426	0.0398

ผลลัพธ์ — **regression (Income)**

ตัววัด	PyTorch	Scratch	ส่วนต่าง
R^2	**0.7423**	**0.7695**	**0.0272**
RMSE ($)	11,064.7	10,463.3	5.4%
MAE ($)	7,067.9	6,543.3	7.4%

4) ทำไมจึงเป็นเช่นนั้น

- Accuracy 0.799 ต่ำกว่า การเดา "ไม่ตอบรับ" ทุกแถว (0.851) เพราะการถ่วงน้ำหนักคลาสบวก (`pos_weight` ≈ 5.7) เพิ่ม recall ขึ้นเป็น 0.72 แลกกับ precision 0.40
- อันดับของลูกค้าถูกขับด้วย `AcceptedCmp1`, `AcceptedCmp2`, `AcceptedCmp3`, `AcceptedCmp4`, `AcceptedCmp5`, `MntWines`, `MntFruits`, `MntMeatProducts`, `MntFishProducts`, `MntSweetProducts`, `MntGoldProds`, `Recency` และ `Customer_Tenure_Days` ตรงกับรายชื่อฟีเจอร์ใน EDA
- regression ได้ R^2 0.74–0.77 สูงตามที่ EDA คาด คอลัมน์ `MntWines`, `MntFruits`, `MntMeatProducts`, `MntFishProducts`, `MntSweetProducts`, `MntGoldProds` เป็นตัวแทนของ `Income` ในเชิงกลไก
- Scratch ได้ R^2 สูงกว่า Library 0.027 ซึ่งเป็นความแปรปรวนจากค่าเริ่มต้น ไม่ใช่ความได้เปรียบของวิธีเขียนเอง

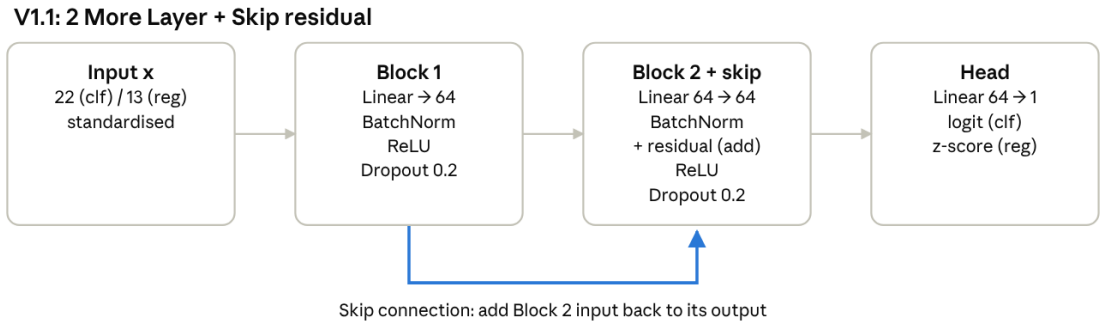
5) คำถามถัดไป

- Loss ลดลงเรื่อย ๆ ไม่มีจุดหัก แสดงว่าโมเดลยัง under-fit
- V1.0 เป็นแค่ baseline ไว้เทียบ ต้องลองทำ MLP ให้มีหลาย Layer แล้วดูว่าคะแนนดีกว่า baseline ไหม → V1.1

4.2 V1.1 — MLP แบบ ResNet

1) คำถาม**:** MLP ที่มีหลาย Layer ให้คะแนนดีกว่า baseline V1.0 (1 hidden layer) หรือไม่ โดยเปลี่ยนแค่จำนวน Layer regularization และ schedule

2) architecture



architecture V1.1

- BatchNorm: ปรับสเกลการเปิดใช้งานของแต่ละหน่วยข้าม batch ทำให้ฝึกนิ่งขึ้น
- Dropout 0.2: สุ่มปิดหน่วยระหว่างฝึก ลด overfitting
- Residual: บวกอินพุตของบล็อกกลับเข้าไป ทำให้เพิ่ม Layer ได้อย่างปลอดภัย
- Adam (lr 1e-3)
- Cosine annealing
- Weight decay 1e-4
- 100 epochs

3) ผลลัพธ์ — **classify (Response)**

ตัววัด	PyTorch	Scratch	ส่วนต่าง
F1	**0.5455**	**0.5158**	**0.0297**
ROC-AUC	0.8818	0.8625	0.0193
Precision	0.4122	0.3984	0.0138
Recall	0.8060	0.7313	0.0747
Accuracy	0.7991	0.7946	0.0045
BCE	0.3966	0.4121	0.0155

ผลลัพธ์ — **regression (Income)**

ตัววัด	PyTorch	Scratch	ส่วนต่าง
R^2	**0.7774**	**0.7595**	**0.0179**
RMSE ($)	10,283.8	10,688.1	3.9%
MAE ($)	6,523.7	6,642.7	1.8%

เทียบกับเวอร์ชันก่อนหน้า **(V1.0 → V1.1)**

ตัววัด	V1.0	V1.1	เปลี่ยน
F1 · PyTorch	0.5161	0.5455	+0.0294
F1 · Scratch	0.4974	0.5158	+0.0184
R^2 · PyTorch	0.7423	0.7774	+0.0351
R^2 · Scratch	0.7695	0.7595	–0.0100

4) ทำไมจึงเป็นเช่นนั้น

- classify: การเพิ่ม Layer ทำให้ recall ของ Library เพิ่มจาก 0.72 เป็น 0.81 โดย precision แทบไม่เสีย (0.40 → 0.41) F1 จึงเพิ่ม
- regression: Library ดีขึ้นชัด (R^2 +0.035, RMSE –7%) แต่ Scratch ลดลง 0.010 ซึ่งอยู่ในช่วงความผันผวนจากค่าเริ่มต้น

- สรุปได้ว่าการเพิ่ม Layer ช่วยให้ดีกว่า baseline V1.0 บน classify และบน regression

5) คำถามถัดไป

- ชั้น 2/3 ที่เก็บไว้มีสัญญาณจริงหรือเป็นแค่น้ำหนักที่ไม่จำเป็น → ทดลองตัดทิ้งใน V1.2

4.3 V1.2 — ตัดชั้น 2/3 เหลือเฉพาะ 3/3

1) คำถาม**:** ฟีเจอร์ชั้น 2/3 (classify 8 ตัว · regression 5 ตัว) มีสัญญาณจริง หรือเป็นแค่น้ำหนักที่ตัดออกได้ (ตามหัวข้อ 2.7 ชั้นนี้ให้พิจารณารายตัว จึงใช้การทดลองตัดสินแทน)

2) architecture: เหมือน V1.1 ทุกประการ (โครงในรูปที่ 4.2, ชุดแบ่ง และ seed เดิม) เปลี่ยนแค่จำนวนอินพุต

- classify: อินพุต 22 → 10 มิติ (ตัด one-hot ของ `Marital_Status` ทิ้ง)
- classify: พารามิเตอร์ 5,185
- classify: ฟีเจอร์ที่ถูกตัด `AcceptedCmp1`, `AcceptedCmp2`, `Marital_Status`, `NumWebPurchases`, `MntSweetProducts`, `MntFishProducts`, `NumStorePurchases`, `Age`
- regression: อินพุต 13 → 8 มิติ
- regression: พารามิเตอร์ 5,057
- regression: ฟีเจอร์ที่ถูกตัด `NumStorePurchases`, `MntFishProducts`, `Kidhome`, `Age`, `NumDealsPurchases`

3) ผลลัพธ์ — **classify (Response)**

ตัววัด	PyTorch	Scratch	ส่วนต่าง
F1	**0.4566**	**0.4645**	**0.0079**
ROC-AUC	0.8452	0.8463	0.0011
Precision	0.3289	0.3403	0.0114
Recall	0.7463	0.7313	0.0150
Accuracy	0.7344	0.7478	0.0134
BCE	0.4712	0.4471	0.0241

ผลลัพธ์ — **regression (`Income`)**

ตัววัด	PyTorch	Scratch	ส่วนต่าง
R^2	**0.7562**	**0.7586**	**0.0024**
RMSE ($)	10,761.2	10,708.1	0.5%
MAE ($)	6,982.8	6,935.8	0.7%

เทียบกับเวอร์ชันก่อนหน้า **(V1.1 → V1.2)**

ตัววัด	V1.1	V1.2	เปลี่ยน
F1 · PyTorch	0.5455	0.4566	–0.0889
F1 · Scratch	0.5158	0.4645	–0.0513
R^2 · PyTorch	0.7774	0.7562	–0.0212
R^2 · Scratch	0.7595	0.7586	–0.0009

4) ทำไมจึงเป็นเช่นนั้น

- classify: F1 ลดลงทิศทางเดียวกันทั้งคู่ ชี้ว่าชั้น 2/3 ส่งผลจริงไม่ซ้ำซ้อน
- feature ที่น่าจะสำคัญที่สุดคือ `AcceptedCmp1` (bivariate 0.294 สูงสุดในที่ถูกตัด) ร่วมกับ non linear feature ของ `Marital_Status`, `NumStorePurchases`, `Age` นี่เป็นข้อสรุปที่ไม่ได้แยกทดสอบทีละตัว

- regression: เมื่อใช้เฉพาะชุด 3/3 (8 ตัว) Library เสีย R^2 เพียง 0.021 และ Scratch แทบไม่เปลี่ยน (–0.001) แสดงว่าฟีเจอร์อีก 5 ตัวที่ถูกตัดเพิ่มข้อมูลให้โมเดลน้อยมาก ชุด 3/3 คือ `MntWines`, `MntMeatProducts`, `MntFruits`, `MntSweetProducts`, `MntGoldProds`, `NumCatalogPurchases`, `NumWebPurchases` และ `NumWebVisitsMonth`
- ความไม่เท่ากันระหว่างสองงานนี้ชี้ว่าความซ้ำซ้อนของฟีเจอร์ regression สูงกว่า classify มาก (ตาม EDA)

5) คำถามถัดไป

- ชั้น 2/3 กลับมาใช้ในทุกโมเดลถัดไป (18 ฟีเจอร์ใน classify และ 13 ฟีเจอร์ใน regression)
- regression ชุด 8 มิติได้ผลใกล้เคียงกันและเล็กกว่า จึงเก็บไว้เป็นทางเลือก แต่ยังใช้ชุด 13 มิติเป็นหลัก
- คำถามใหม่: architecture นอกตำราให้คะแนนบนชุดฟีเจอร์เดียวกันได้ดีกว่าหรือไม่ → V2.0

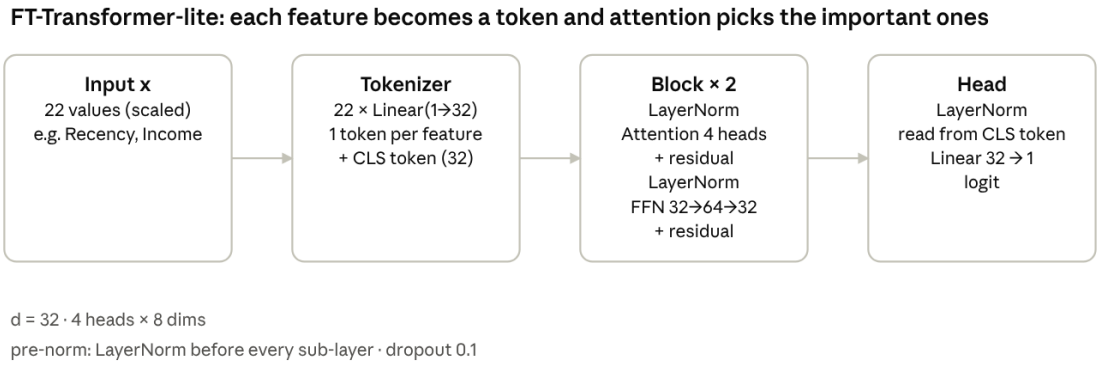
4.4 V2.0 — architecture นอกตำรา: FT-Transformer-lite และ NAM

1) คำถาม**:** architecture ที่ไม่อยู่ในรายชื่อที่เรียน (ไม่ใช่ Linear/Logistic, Tree, RF, GB, NB, SVM, kNN, k-means, Perceptron, MLP) จะให้คะแนนดีกว่า MLP หลาย Layer หรือไม่ และจะช่วยอธิบายผลได้หรือไม่

- classify: **FT-Transformer-lite**
- regression: **NAM** (Neural Additive Model) พร้อม `log1p` + Huber (ทุกฟีเจอร์มี neural network เล็ก ๆ ของตัวเองแล้วรวมผล)

2) architecture และการทำงาน

4.4.1 FT-Transformer-lite (classify)



architecture V2.0 (classify)

การทำงานทีละขั้น:

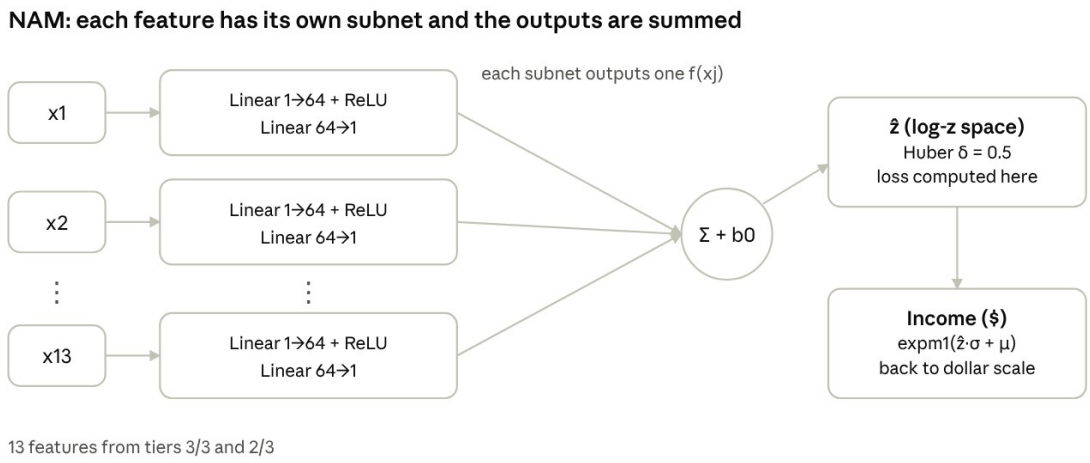
1. **Tokenizer:** แต่ละฟีเจอร์ x_j ผ่าน Linear(1→32) ของตัวเอง ได้เวกเตอร์ 32 มิติ
2. ได้ 22 token ตามจำนวนฟีเจอร์ บวก CLS token ที่เรียนรู้ได้ 1 ตัว รวม 23 token
3. **Self-attention (4 หัว):** ทุก token สร้าง Q, K, V ด้วย Linear 32→32
4. คะแนนความสัมพันธ์กับ token อื่นทั้งหมดผ่าน softmax (รวมเป็น 1)
5. เฉลี่ย V ตามน้ำหนักนั้น
6. **FFN:** แต่ละ token ผ่าน Linear 32→64, ReLU, Linear 64→32
7. **Residual:** ทุก sub-layer บวกอินพุตกลับเข้าไป
8. ซ้ำ 2 บล็อก
9. **Head:** ใช้เฉพาะ CLS token สุดท้าย
10. ผ่าน LayerNorm
11. ผ่าน Linear 32→1 ได้ logit

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V, \qquad d_h = \frac{32}{4} = 8$$

- แถวของ CLS ในเมทริกซ์น้ำหนักคือ “CLS อ่านสัญญาณจากแต่ละฟีเจอร์เท่าไร” จึงใช้เป็นค่าความสำคัญของฟีเจอร์ได้
- พารามิเตอร์รวม: tokenizer 1,408 + CLS 32 + 2 บล็อก × 8,544 + LayerNorm สุดท้าย 64 + head 33 = 18,625
- Weighted BCE (`pos_weight` ≈ 5.7)

- Adam (lr 1e-3) + cosine
- Weight decay 1e-4
- 100 epochs
- เก็บ checkpoint ที่ F1 ดีที่สุด

4.4.2 NAM + log1p + Huber (regression)



architecture V2.0 (regression)

การทำงานทีละขั้น:

1. **Subnet** ต่อฟีเจอร์**:** แต่ละฟีเจอร์ x_j ผ่าน neural network เล็กของตัวเอง f_j (Linear 1→64, ReLU, Linear 64→1)
2. ไม่มี subnet ใดเห็นฟีเจอร์อื่น
3. รวมด้วยการบวก**:** ẑ = b0 + Σ f_j(x_j)
4. แยกอ่านอิทธิพลของแต่ละฟีเจอร์ได้ตรง ๆ (glass-box)
5. เป้าหมายที่แปลงแล้ว**:** ฝึกกับ z-score ของ log1p(Income)
6. เหตุผล: รายได้เบ้ขวา (สูงสุด $162,397 เทียบค่าเฉลี่ย $51,959)
7. **Huber δ = 0.5:** ความคลาดเล็กคิดแบบกำลังสอง ความคลาดใหญ่คิดแบบเชิงเส้น
8. แถวรายได้สูงจึงดึงการฝึกน้อยกว่า MSE
9. คืนสเกล**:** Income = expm1(ẑ·σ + μ) เป็นหน่วยดอลลาร์
10. คำนวณตัวชี้วัดทุกตัวหลังคืนสเกล

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2 & |e| \le \delta \\ \delta\left(|e| - \frac{1}{2}\delta\right) & |e| > \delta \end{cases}, \qquad \delta = 0.5$$

- พารามิเตอร์รวม 2,510 = 13 subnet × (64 + 64 + 64 + 1) + b0
- Adam (lr 1e-3) + cosine
- Weight decay 1e-4
- 100 epochs
- เก็บ checkpoint ที่ RMSE ต่ำสุด

3) ผลลัพธ์ — **classify (`Response`, FT-Transformer-lite)**

ตัววัด	PyTorch	Scratch	ส่วนต่าง
F1	**0.6038**	**0.5197**	**0.0841**
ROC-AUC	0.8657	0.8251	0.0406
Precision	0.5217	0.5500	0.0283
Recall	0.7164	0.4925	0.2239
Accuracy	0.8594	0.8638	0.0044
BCE	0.4007	0.4278	0.0271

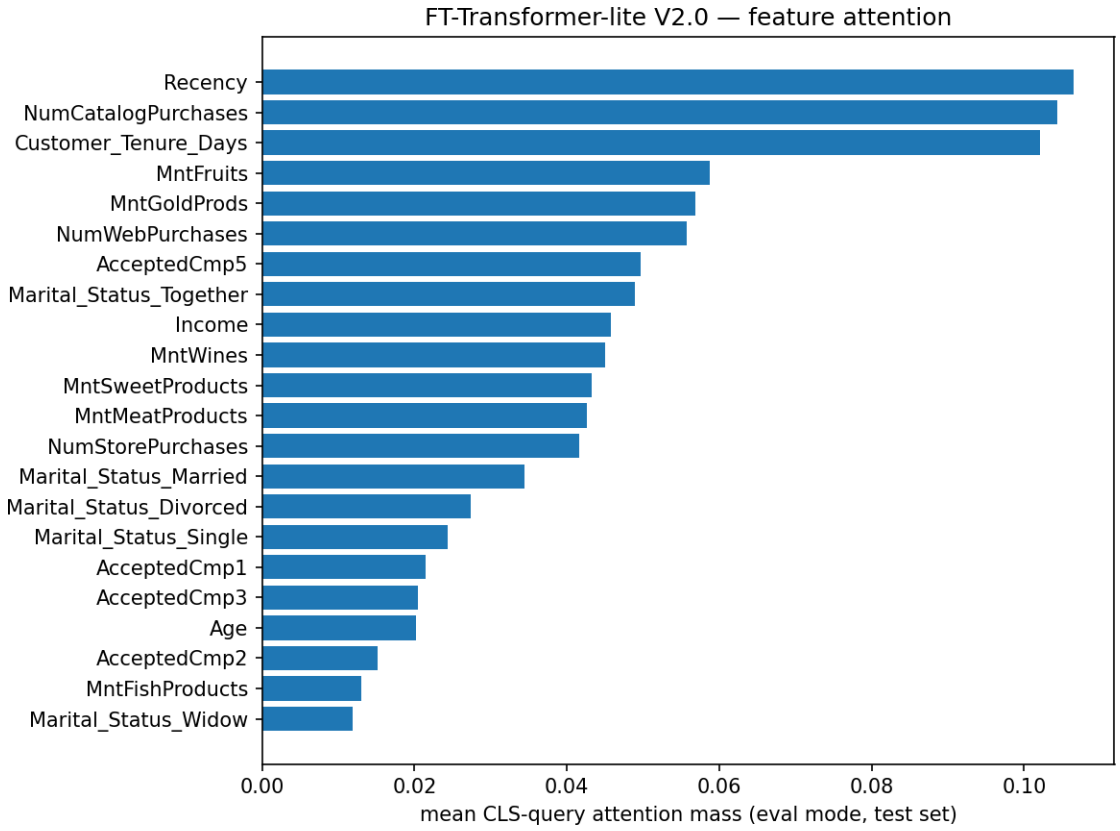
ผลลัพธ์ — **regression (`Income`, NAM)**

ตัววัด	PyTorch	Scratch	ส่วนต่าง
R^2	**0.7233**	**0.7286**	**0.0053**
RMSE ($)	11,463.5	11,353.3	1.0%
MAE ($)	7,186.2	7,064.1	1.7%

เทียบกับเวอร์ชันก่อนหน้าได้ไม๊ใช่ **V1.2** แต่คือ **V1.1 (**เพราะ **V1.2** ใช้ชุดฟีเจอร์เล็กกว่า**)**

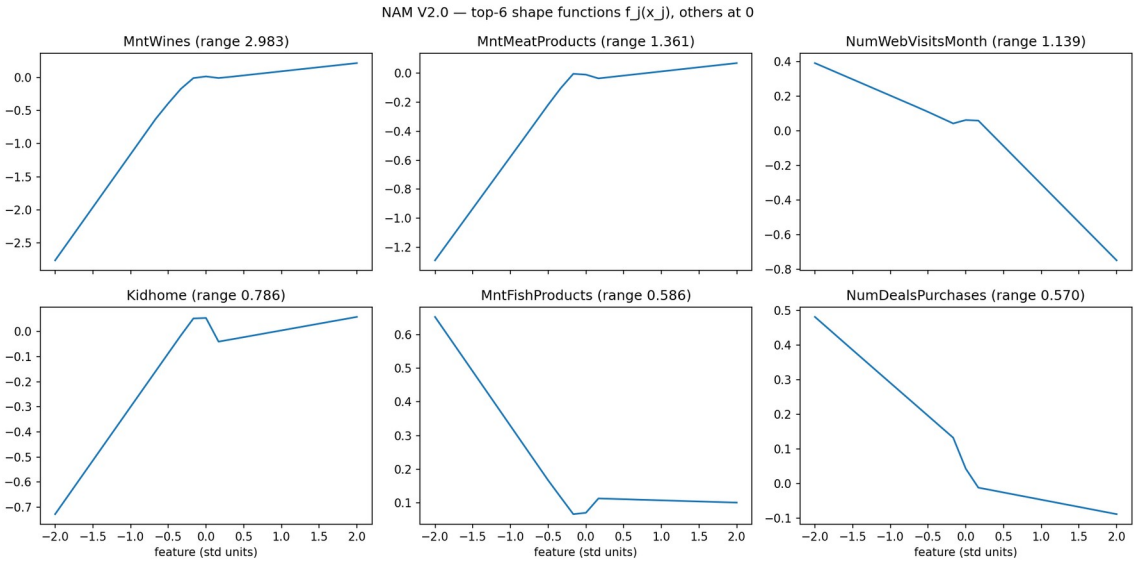
ตัววัด	V1.1	V2.0	เปลี่ยน
F1 · PyTorch	0.5455	0.6038	+0.0583
F1 · Scratch	0.5158	0.5197	+0.0039
R^2 · PyTorch	0.7774	0.7233	−0.0541
R^2 · Scratch	0.7595	0.7286	−0.0309

ภาพ: **CLS-query attention** ของ **FT-Transformer-lite (Library,** ชุดทดสอบ**)**



- 3 ฟีเจอร์บนสุดคือ Recency (0.107), NumCatalogPurchases (0.104) และ Customer_Tenure_Days (0.102)
- Recency และ Customer_Tenure_Days ตรงกับอันดับ 1–2 ของ RF (หัวข้อ 2.6) ไม่ใช่อันดับ bivariate ที่เป็น AcceptedCmp1, AcceptedCmp2, AcceptedCmp3, AcceptedCmp4, AcceptedCmp5

ภาพ: **shape function** ของ **NAM (6** ฟีเจอร์ที่ช่วงผลใหญ่สุด**)**



- MntWines มีช่วงผลใหญ่ที่สุด (2.983) รองลงมาคือ MntMeatProducts (1.361)
- MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds แต่ละตัวสูงขึ้น → รายได้ทำนายสูงขึ้น แต่กราฟจะชันที่ฝั่งค่าต่ำและแบนลงทางฝั่งสูง
- NumWebVisitsMonth และ NumDealsPurchases เป็นเส้นลดลง ตรงกับความสัมพันธ์เชิงลบใน EDA

4) ทำไมจึงเป็นเช่นนั้น

- classify: F1 ของ Library สูงกว่า V1.1 อยู่ 0.058 แต่ค่านี้ได้จาก checkpoint ที่เลือกบนชุดทดสอบ (กล่องข้างบน) จึงยังไม่ใช่หลักฐานว่า attention ชนะ MLP Scratch ใน classify แทบไม่ขยับ (+0.004)
- ความต่างระหว่างสองฝั่ง 0.084 เกินเกณฑ์ 0.05 เป็นครั้งแรกที่ parity ไม่ผ่าน เพราะข้อมูลมีน้อย (ฝึก 1,788 แถว ลูกค้าที่ตอบรับในชุดทดสอบแค่ 67 แถว) ทำให้ผลแกว่งง่าย แม้ BCE ของสองฝั่งจะต่างกันแค่ 0.027
- regression: NAM แพ้ V1.1 ทั้งสองฝั่ง (R^2 –0.054 และ –0.031) เพราะ NAM ดูแต่ละฟีเจอร์แยกกันแล้วค่อยบวกผล จึงใช้ข้อมูลจาก MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds พร้อมกัน ไม่ได้ ส่วน MLP หลาย Layer ใช้ได้
- ถ้าตัด 9 แถวที่ Income สูงสุดในชุดทดสอบออก NAM จะดีขึ้นมาก (RMSE $8,070, R^2 0.835) แสดงว่า NAM พลาดส่วนใหญ่ที่ลูกค้ารายได้สูงมาก ตัวเลขนี้ใช้ดูสาเหตุเท่านั้น ไม่ใช่คะแนนหลัก

5) คำถามถัดไป

- V1.1 ยังเป็นค่าอ้างอิงที่สูงที่สุดใน regression และคะแนน classify ของ V2.0 ไวต่อการเลือก checkpoint
- คำถามใหม่: ข้อมูลนี้ทำคะแนนได้สูงสุดเท่าไร โดยใช้ gradient boosting เป็นตัวเทียบ → V3.0

4.5 V3.0 — Gradient Boosting (LightGBM)

1) คำถาม**:** neural network อาจไม่ใช่วิธีที่เหมาะกับข้อมูลนี้ ลองใช้ gradient boosting แทนดีไหม

- ชุดข้อมูล ฟีเจอร์ (18 ตัวใน classify, 13 ตัวใน regression) และ split เหมือน V1.1 ทุกประการ ชุดทดสอบ 448 แถว (classify) และ 443 แถว (regression) เหมือนเดิม

2) architecture และกลไก

1. เริ่มจากค่าคงที่ F0
2. ทุกรอบคำนวณ gradient g ของ loss ต่อแถว
3. ทุกรอบคำนวณ Hessian h ของ loss ต่อแถว
4. สร้างต้นไม้ CART ลึกไม่เกิน 5 ชั้น (ใบละอย่างน้อย 20 แถว)
5. เลือกจุดแบ่งที่ให้ gain สูงสุด
6. คำนวณค่าใบไม้แบบ Newton step
7. บวกค่าใบไม้เข้าคำทำนายสะสม คูณ learning rate 0.05
8. วนกลับไปข้อ 2

$$\text{gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right], \qquad w_{\text{leaf}} = -\frac{G}{H+\lambda}$$

- classify: F0 = logit ของอัตราบวก ≈ −1.74
- classify: g = w·(p − y)
- classify: h = w·p(1 − p)
- classify: w = `scale_pos_weight` (5.697) สำหรับลูกค้าที่ตอบรับ และ 1 สำหรับลูกค้าที่ไม่ตอบรับ
- regression: F0 = ค่าเฉลี่ยของ `Income`

- regression: g = ค่าทำนาย – y, h = 1 (กลายเป็นการฟิตค่าคลาดเคลื่อนผ่านกลไกเดียวกัน)
- regression: เทรนบน `Income` ดิบ ไม่ทำ scaling (ต้นไม้ไม่ไวต่อสเกล)
- Library (LightGBM): `learning_rate` 0.05
- Library (LightGBM): `num_leaves` 31
- Library (LightGBM): subsample 0.8
- Library (LightGBM): colsample 0.8
- Library (LightGBM): `reg_lambda` 1.0
- Scratch: ต้นไม้ depth-wise ลึก 5 (แทน leaf-wise ของ LightGBM)
- Scratch: min leaf 20
- Scratch: พารามิเตอร์อื่นเหมือน Library
- Early stopping ทำบน 10% ที่เอาออกจากชุด **train** (patience 50)
- ไม่ใช้ชุดทดสอบ ซึ่งต่างจาก V2.0 ที่เลือก checkpoint บนชุดทดสอบ
- Sanity check: ต้นไม้ลึก 2 ตรงกับ `sklearn.tree.DecisionTreeRegressor` (ต่างสูงสุด 4×10^{-16})
- Sanity check: ตัวอย่าง boosting 3 แถว 2 รอบที่คำนวณมือได้ผลเท่ากัน
- Sanity check: loss ลดลงทุกรอบใน 20 รอบแรก

3) ผลลัพธ์ — **classify (`Response`)**

ตัววัด	LightGBM	Scratch	ส่วนต่าง
F1	**0.6014**	**0.0000**	**0.6014**
ROC-AUC	0.8898	0.8310	0.0588
Precision	0.5658	0.0000	0.5658
Recall	0.6418	0.0000	0.6418
Accuracy	0.8728	0.8504	0.0224
BCE	0.2967	0.3757	0.0790

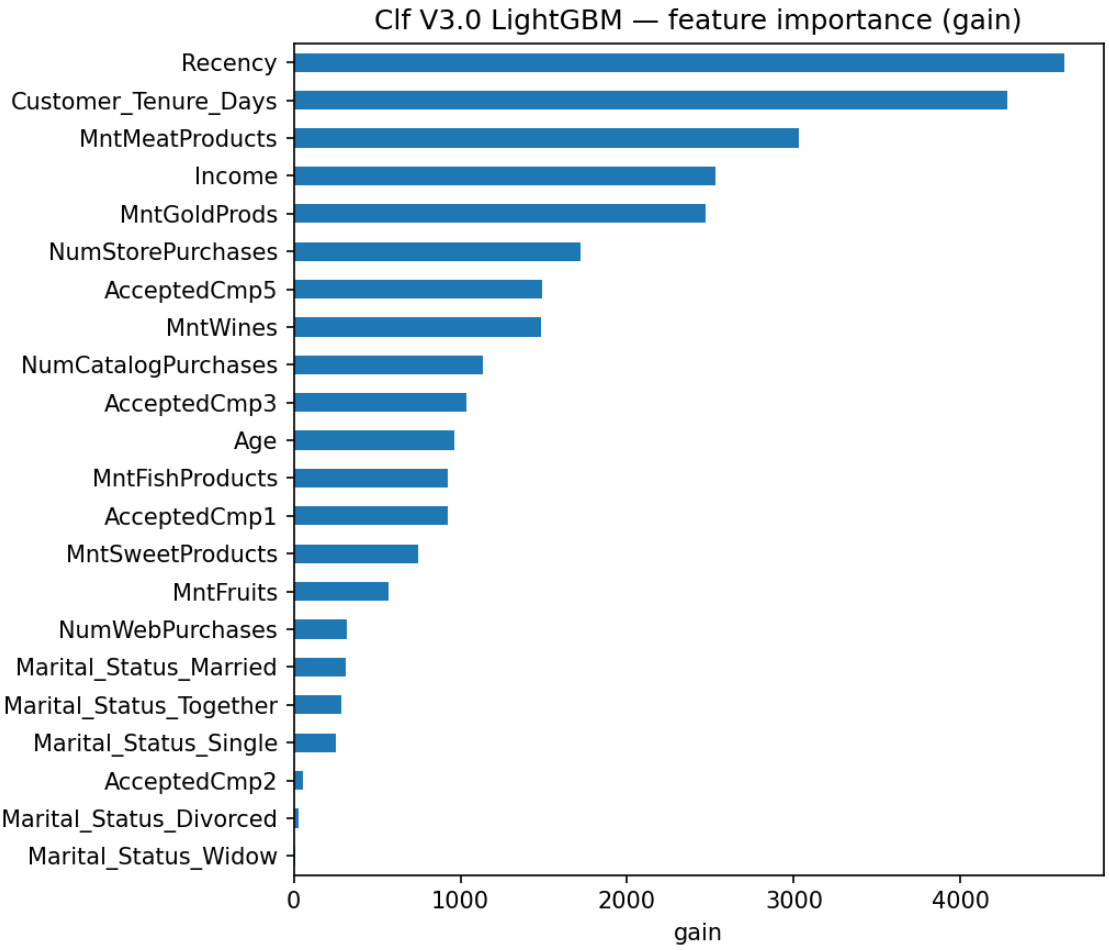
ผลลัพธ์ — **regression (`Income`)**

ตัววัด	LightGBM	Scratch	ส่วนต่าง
R^2	**0.8212**	**0.8213**	**0.0001**
RMSE ($)	9,216.7	9,212.9	0.04%
MAE ($)	5,758.4	5,904.9	2.5%

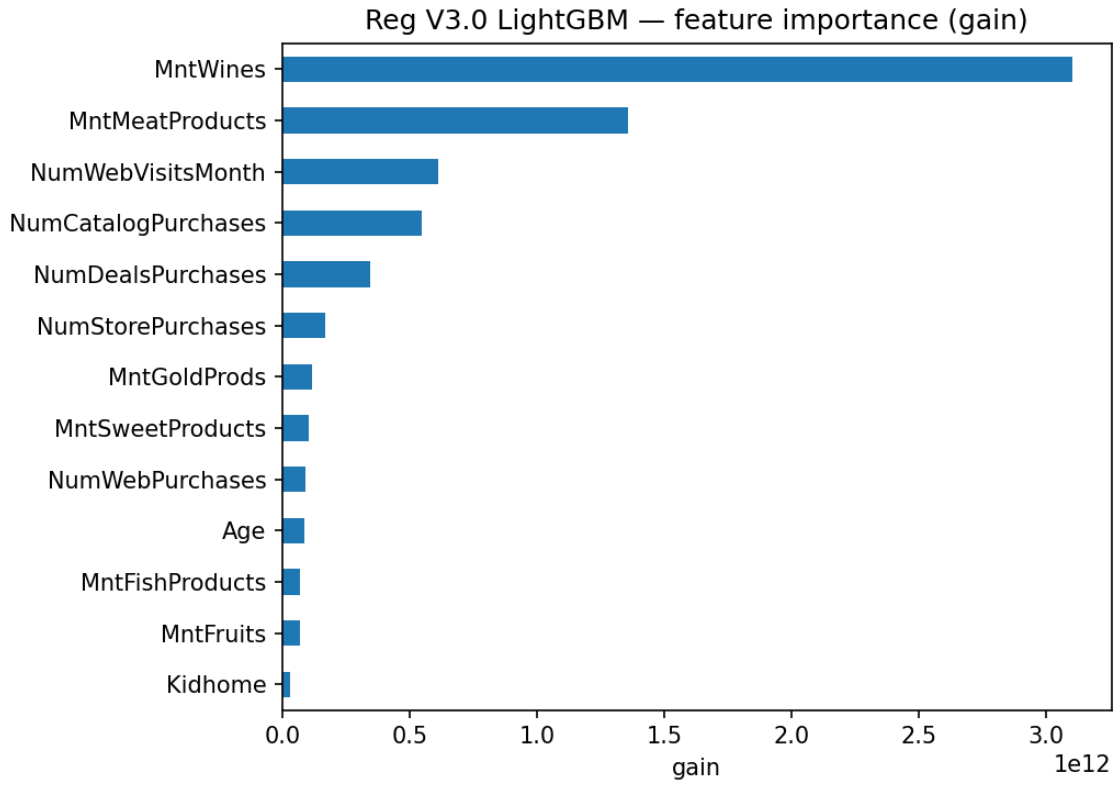
เทียบกับเวอร์ชันก่อนหน้า **(V2.0 → V3.0)**

ตัววัด	V2.0	V3.0	เปลี่ยน
F1 · Library	0.6038	0.6014	–0.0024
F1 · Scratch	0.5197	0.0000	–0.5197
R^2 · Library	0.7233	0.8212	+0.0979
R^2 · Scratch	0.7286	0.8213	+0.0927

ภาพ**: gain-based feature importance** ของ **LightGBM — classify**



ภาพ: gain-based feature importance ของ LightGBM — regression



4) ทำไมจึงเป็นเช่นนั้น

- regression: R^2 0.821 สูงกว่า V1.1 (0.777) อยู่ 0.044 และสองฝั่งได้ผลเท่ากัน (ต่าง 0.0001) ซึ่งเป็นสัมพันธ์ชัดที่สุดในโครงงาน แสดงว่าข้อมูลนี้เหมาะกับโมเดลแบบต้นไม้ (tree-based) มากกว่า neural network
- ฟีเจอร์ regression อันดับบนคือ `MntWines` และ `MntMeatProducts` ตามด้วย `NumWebVisitsMonth`, `NumCatalogPurchases`, `NumDealsPurchases` สอดคล้องกับเรื่อง “`MntWines`, `MntFruits`, `MntMeatProducts`, `MntFishProducts`, `MntSweetProducts`, `MntGoldProds` เป็นตัวแทนของ `Income`” ใน EDA
- classify: F1 ของ LightGBM (0.601) สูงกว่า V1.1 (0.546) อยู่ 0.056 และใกล้ V2.0 Library มาก (0.604) ทั้ง AUC 0.890 สูงที่สุดในทุกโมเดล
- `Recency` และ `Customer_Tenure_Days` อยู่อันดับ 1–2 เหมือน RF และ attention ของ FT-Transformer-lite สามวิธีชี้ไปทางเดียวกัน: สัญญาณจาก `Recency`/`Customer_Tenure_Days` แข็งกว่าสัญญาณจาก `AcceptedCmp1`, `AcceptedCmp2`, `AcceptedCmp3`, `AcceptedCmp4`, `AcceptedCmp5` โดยตรง

- **Parity classify fail:** Scratch ได้ F1 = 0 เพราะ early stopping บน BCE หยุดที่รอบ 8 ซึ่งความน่าจะเป็นทำนายทุกแถวให้อยู่ในช่วง 0.12–0.19 ไม่เคยเกิน threshold 0.5 แม้ ROC-AUC (0.831) ยังใกล้ LightGBM
- กลไกเกิดจากความไม่สมดุล: ค่าเริ่มต้น F0 ≈ −1.74 ทำให้ BCE ดีอยู่แล้วเมื่อทำนายใกล้อัตราฐาน ในขณะที่ learning rate เล็ก (0.05) ยังไม่ทันยกคะแนนแถวใดขึ้นไปเหนือศูนย์
- การตัดแถวเดียวออกจากชุด train ย้ายจุด BCE ต่ำสุดจากรอบที่ 254 ไปรอบที่ 8 แสดงว่าจุดหยุดไวต่อการแบ่งข้อมูลมาก

5) คำถามถัดไป

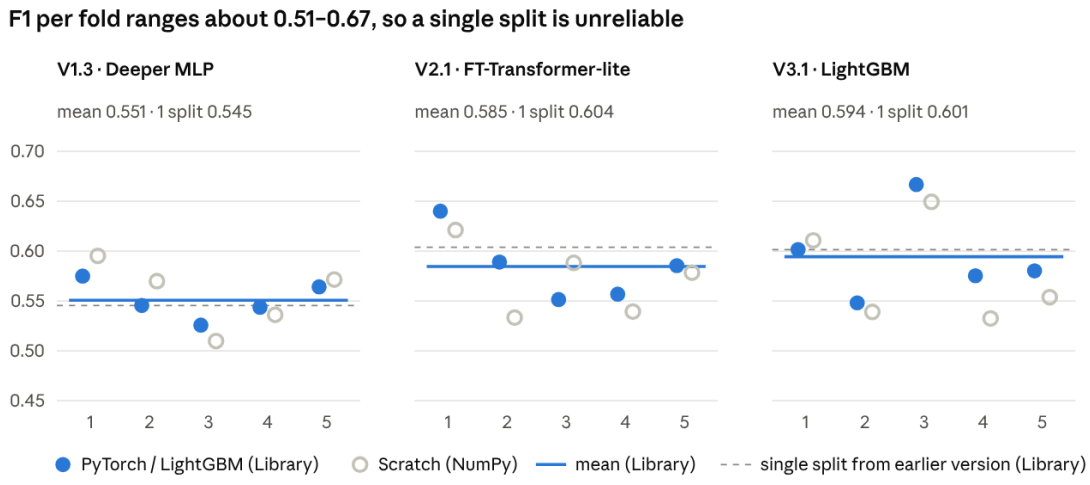
- คะแนนจาก split เดียวไม่น่าเชื่อพอ เพราะชุดทดสอบมีลูกค้าที่ตอบรับแค่ 67 แถว → รัน 5-fold ให้ในทุกครอบครัวโมเดล (V1.3, V2.1, V3.1)

4.6 V1.3, V2.1, V3.1 — ตรวจซ้ำด้วย 5-fold cross-validation

1) คำถาม: ผลจาก split 80/20 เดียวเป็นตัวแทนที่ดีหรือเป็นแค่การสุ่มที่โชคดี/ร้าย โดยเฉพาะจุดที่น่าสงสัยคือ F1 0.6038 ของ V2.0 และ F1 = 0 ของ V3.0 scratch

2) วิธีทำ: ใช้ architecture และ hyper-parameter เดิมคัดลอกจาก V1.1, V2.0, V3.0 ไม่แก้ แล้วห่อด้วย 5-fold (รายละเอียดในหัวข้อ 3.6)

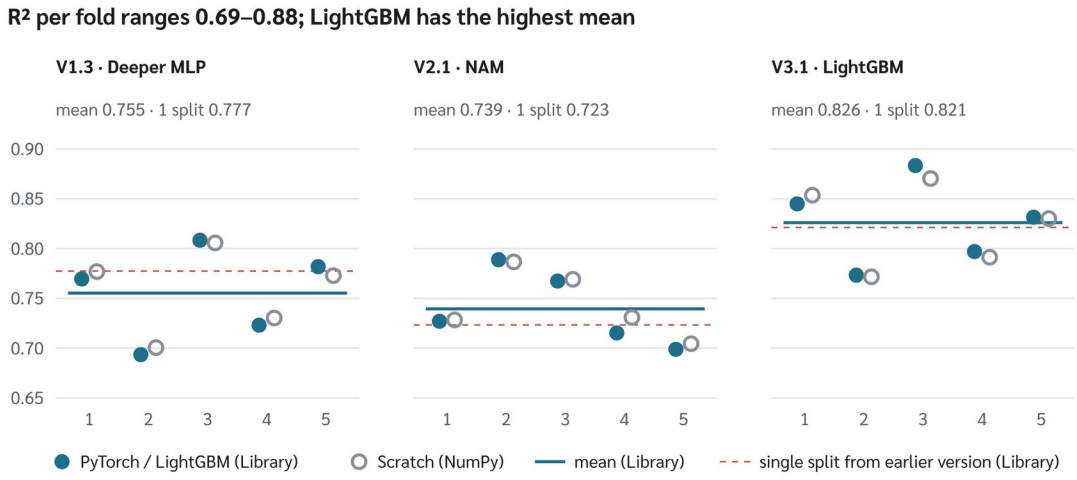
3) ผลลัพธ์ — **classify (F1 ต่อ fold)**



V1.3, V2.1, V3.1 (classify, StratifiedKFold k=5, seed 42) · y-axis starts at 0.45

เวอร์ชัน	PyTorch / LightGBM (mean ± SD)	Scratch (mean ± SD)	split เดียว (Library)
V1.3 MLP หลาย Layer	0.5508 ± 0.0191	0.5565 ± 0.0335	0.5455
V2.1 FT-Transformer-lite	0.5845 ± 0.0352	0.5720 ± 0.0363	0.6038
V3.1 LightGBM	0.5944 ± 0.0447	0.5771 ± 0.0509	0.6014

ผลลัพธ์ — **regression (R^2 ต่อ fold)**



V1.3, V2.1, V3.1 (regression, KFold k=5, seed 42) · y-axis starts at 0.65

เวอร์ชัน	PyTorch / LightGBM (mean ± SD)	Scratch (mean ± SD)	split เดียว (Library)
V1.3 MLP หลาย Layer	0.7553 ± 0.0463	0.7573 ± 0.0416	0.7774
V2.1 NAM	0.7395 ± 0.0375	0.7439 ± 0.0333	0.7233
V3.1 LightGBM	0.8260 ± 0.0427	0.8234 ± 0.0414	0.8212

4) ทำไมจึงเป็นเช่นนั้น

- ความแกว่งระหว่าง fold (SD ราว 0.02–0.05 ใน F1 และ R^2) ใหญกว่าส่วนต่างระหว่าง architecture หลายคู่ของค่าเฉลี่ย

- classify: ค่าที่ V2.0 รายงาน (0.6038) สูงกว่าค่าเฉลี่ย 5-fold (0.5845) จึงเป็น split ที่โชคดี ส่วน V3.0 (0.6014) และ V1.1 (0.5455) อยู่ใกล้ค่าเฉลี่ย
- Parity fail ของ V2.0 ไม่เกิดซ้ำ: ส่วนต่างค่าเฉลี่ย Library–scratch เหลือ 0.0125 และทุก fold ต่างกันไม่เกิน 0.06 ซึ่งยืนยันว่าความต่าง 0.084 ใน V2.0 มาจากการเลือก epoch ที่ไม่เท่ากัน ไม่ใช่การคำนวณที่ต่างกัน
- F1 = 0 ของ V3.0 scratch ไม่เกิดใน fold ใดเลย (best_iter 235–369 ทุก fold) เป็นผลจากจุดหยุด early-stopping ของ split เดียว ไม่ใช่ข้อบกพร่องอย่างเป็นระบบของวิธี
- ถ้าเทียบทีละ fold (ทุกเวอร์ชันใช้ seed 42 และวิธีแบ่งเดียวกัน) FT-Transformer-lite ได้ F1 สูงกว่า MLP หลาย Layer ทั้ง 5 จาก 5 fold (+0.013 ถึง +0.065, เฉลี่ย +0.034) แต่ต้องระวัง: V2.1 เลือก checkpoint บน fold ที่ใช้รายงาน ส่วน V1.3 ฝึกคงที่ 100 epochs ช่องว่างบางส่วนจึงมาจากวิธีเลือก checkpoint
- LightGBM ไม่ได้ชนะ FT-Transformer-lite อย่างสม่ำเสมอใน classify (สูงกว่า 2 จาก 5 fold เท่านั้น ผลต่างเฉลี่ย 0.010 เทียบกับ SD 0.04) จึงบอกได้แค่ว่าใกล้เคียงกัน
- regression: LightGBM ได้ R^2 สูงกว่า MLP หลาย Layer ทุก fold (+0.05 ถึง +0.08) และสูงกว่า NAM 4 จาก 5 fold (NAM ชนะใน fold 2) ผลสรุปก่อนหน้า “regression เหมาะกับ GBDT ที่สุด” จึงยังเป็นจริงบน 5-fold
- NAM แพ้ MLP หลาย Layer ที่ค่าเฉลี่ย 0.016 แต่ไม่ทุก fold (ชนะเฉพาะ fold 2) ราคาของความอธิบายได้จึงผันแปรตาม fold ไม่ใช่จำนวนคงที่

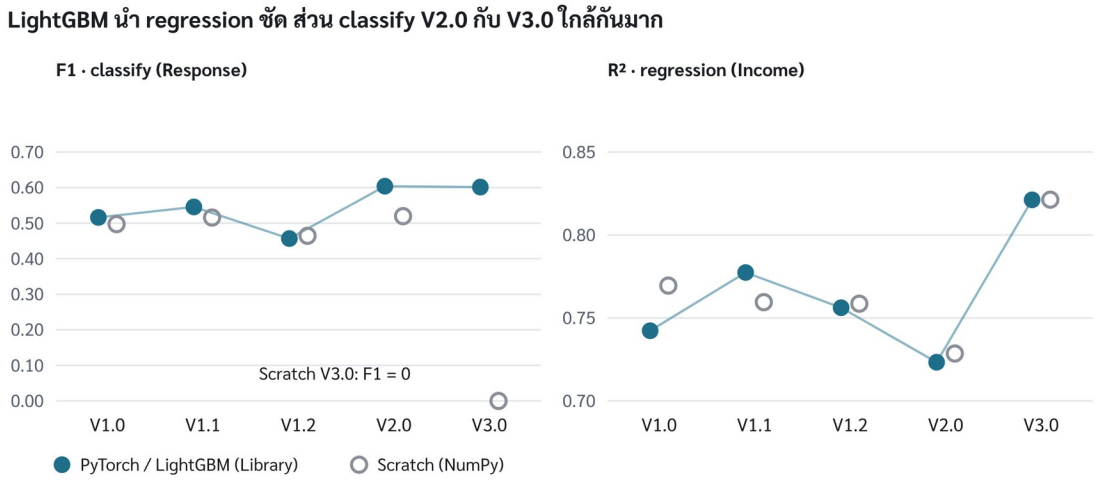
5) คำถามถัดไป

- ไม่มี validation set แยกในทุกเวอร์ชันที่เลือก checkpoint บน fold/test จึงควรบันทึกเป็นข้อจำกัด (บทที่ 5) และใช้ค่า mean ± SD เป็นตัวเลขหลักในการเปรียบเทียบ

บทที่ **5** ผลการทดลองและการเปรียบเทียบ

บทนี้รวบผลจากบทที่ 4 มาเรียงในกระดานเดียว แล้วถามว่าความแตกต่างแต่ละจุดเชื่อได้แค่ไหน

5.1 กระดานคะแนนรวมทุกเวอร์ชัน (split เดียว)



V1.0–V3.0, test split 20% (n = 448 classify, 443 regression, seed 42) · แกน y ของแต่ละฝั่งคนละช่วง

classify — F1 (บน Response)

เวอร์ชัน	PyTorch / LightGBM	Scratch	ส่วนต่าง
V1.0 MLP ตื้น	0.5161	0.4974	0.0187
V1.1 MLP หลาย Layer	0.5455	0.5158	0.0297
V1.2 ตัดฟีเจอร์ 3/3	0.4566	0.4645	0.0079
V2.0 FT-Transformer-lite	**0.6038**	0.5197	0.0841
V3.0 LightGBM	0.6014	0.0000	0.6014

regression — R² (บน Income)

เวอร์ชัน	PyTorch / LightGBM	Scratch	ส่วนต่าง
V1.0 MLP ตื้น	0.7423	0.7695	0.0272
V1.1 MLP หลาย Layer	0.7774	0.7595	0.0179
V1.2 ตัดฟีเจอร์ 3/3	0.7562	0.7586	0.0024
V2.0 NAM	0.7233	0.7286	0.0053
V3.0 LightGBM	**0.8212**	**0.8213**	0.0001

regression — RMSE ($, Sub Metrics)

เวอร์ชัน	PyTorch / LightGBM	Scratch	ส่วนต่าง
V1.0 MLP ตื้น	11,064.7	10,463.3	601.4
V1.1 MLP หลาย Layer	10,283.8	10,688.1	404.3
V1.2 ตัดฟีเจอร์ 3/3	10,761.2	10,708.1	53.1
V2.0 NAM	11,463.5	11,353.3	110.2
V3.0 LightGBM	**9,216.7**	**9,212.9**	3.8

- Baseline แบบง่าย: ทำนายทุกแถวว่า “ไม่ตอบรับ” ได้ accuracy 0.8504 (381 จาก 448 แถว) แต่ F1 = 0 จึงใช้ F1 เป็นตัวตัดสินหลักทั้งบท
- ทุกโมเดลในตารางทำนายเก่งกว่าการทำนายค่าเฉลี่ย ($R^2 = 0$) อย่างชัดเจน แต่ช่วง R^2 0.72–0.82 ชี้ว่าความแตกต่างอยู่ที่ชนิดโมเดล
- classify F1 สูงสุดคือ V2.0 (0.6038) แต่ใกล้ V3.0 (0.6014) มาก จะตัดสินได้ควรดูผล 5-fold ในหัวข้อ 5.3

5.2 เทียบกับเวอร์ชันก่อนหน้า

การเปลี่ยน	Δ F1 (Library)	Δ R^2 (Library)
V1.0 → V1.1 (หลาย Layer + ResNet)	+0.0294	+0.0351
V1.1 → V1.2 (ตัดชั้น 2/3)	–0.0889	–0.0212
V1.1 → V2.0 (FT-Transformer-lite / NAM)	+0.0583	–0.0541
V1.1 → V3.0 (LightGBM)	+0.0559	+0.0438

- การปรับที่ได้ผลทั้งสองงานมีสองจุด: V1.0 → V1.1 และ V1.1 → V3.0
- V1.2 แย่ลงทั้งสองงาน ยืนยันว่าฟีเจอร์ชั้น 2/3 มีสัญญาณจริง (สอดคล้องกับหัวข้อ 2.7 ที่ให้พิจารณาชั้นนี้รายตัว)
- V2.0 สลับทิศ: architecture ใหม่ช่วย classify แต่ทำให้ regression แย่ลง (NAM ยอมแลกความแม่นยำกับความอธิบายได้)
- V3.0 ใน classify ไม่แซง V2.0 (–0.0024) แต่สูงกว่า V1.1 ชัดเจน

5.3 ผล **5-fold:** ลำดับที่ทนต่อโชคของ **split**

โมเดล	F1 (mean ± SD)	R^2 (mean ± SD)
V1.3 MLP หลาย Layer	0.5508 ± 0.0191	0.7553 ± 0.0463
V2.1 FT-Transformer-lite / NAM	0.5845 ± 0.0352	0.7395 ± 0.0375
V3.1 LightGBM	**0.5944 ± 0.0447**	**0.8260 ± 0.0427**

(ค่าของฝั่ง PyTorch / LightGBM เท่านั้น ผล Scratch อยู่ในหัวข้อ 4.6)

ค่าจาก **split** เดียวเทียบค่าเฉลี่ย **5-fold (Library)**

โมเดล · ตัววัด	split เดียว	5-fold mean	ส่วนต่าง
MLP หลาย Layer · F1	0.5455	0.5508	–0.0053
FT-Transformer-lite · F1	0.6038	0.5845	+0.0193
LightGBM · F1	0.6014	0.5944	+0.0070
MLP หลาย Layer · R^2	0.7774	0.7553	+0.0221
NAM · R^2	0.7233	0.7395	–0.0162
LightGBM · R^2	0.8212	0.8260	–0.0048

- ทุกคู่ไม่เกิน ±0.025 จึงไม่มีจุดใดในกระดาน 5.1 ที่เป็นการสุ่มที่โชคดีหรือร้ายจนทำให้อันดับผิด จุดที่ “ดีเกินจริงที่สุด” คือ F1 ของ V2.0 (+0.019)
- regression: LightGBM นำอันดับในทุก fold เทียบกับ MLP หลาย Layer แม้ผลต่างของค่าเฉลี่ย (+0.071) จะใกล้ SD ของ fold (0.04–0.05) ก็ตาม เพราะ fold เดียวกันทุกโมเดลยากพอกัน การเทียบทีละคู่จึงสะอาดกว่าการเทียบ SD รวม
- classify: สองอันดับบน (V3.1 กับ V2.1) ต่างกัน 0.010 ซึ่งเล็กกว่า SD มาก จึงสรุปได้แค่ว่าทั้งสองแซง MLP หลาย Layer (ทั้ง 5 จาก 5 fold ทั้งสำหรับ FT-Transformer-lite และ LightGBM)

5.4 Parity: PyTorch / LightGBM กับ Scratch

เวอร์ชัน (split เดียว)	Δ F1 classify	ผลตามเกณฑ์ 0.05	Δ R^2 regression
V1.0	0.0187	ผ่าน	0.0272
V1.1	0.0297	ผ่าน	0.0179
V1.2	0.0079	ผ่าน	0.0024
V2.0	0.0841	ไม่ผ่าน	0.0053
V3.0	0.6014	ไม่ผ่าน	0.0001

เวอร์ชัน (5-fold)	ส่วนต่างค่าเฉลี่ย F1	ส่วนต่างค่าเฉลี่ย R^2
V1.3	0.0057	0.0020
V2.1	0.0125	0.0044
V3.1	0.0173	0.0026

- parity ไม่ผ่านเกิดใน classify split เดียวสองครั้ง (V2.0, V3.0) และ regression parity แนบในทุกเวอร์ชัน
- ทั้งสองครั้งมาจากการเลือกจุดหยุดฝึก (เลือก epoch ที่ไม่เท่ากันใน V2.0 · early stopping บน BCE หยุดที่รอบ 8 ใน V3.0) ไม่ใช่สูตรคณิตศาสตร์: V3 ผ่าน sanity check
- ผล 5-fold ยืนยัน: ส่วนต่างเฉลี่ยทุกคู่อยู่ในเกณฑ์ (≤0.018) และ scratch ไม่เกิด F1 = 0 ใน fold ใดเลย
- ผล regression (Δ R^2 ไม่เกิน 0.03 ทุกครั้ง) ทำให้เชื่อได้ว่า neural network และ GBDT ที่เขียนเองด้วย NumPy ทำงานเทียบเท่าไลบรารีอ้างอิง

5.5 ฟีเจอร์ที่หลายวิธีชี้ตรงกัน

- classify: `Recency` และ `Customer_Tenure_Days` อยู่อันดับบนทั้งใน RF (หัวข้อ 2.6) gain ของ LightGBM และ attention ของ FT-Transformer-lite ตามด้วย `MntMeatProducts`, `Income`, `MntGoldProds`
- regression: `MntWines` และ `MntMeatProducts` นำ ตามด้วย `NumWebVisitsMonth`, `NumCatalogPurchases`, `NumDealsPurchases` ซึ่งเป็นพฤติกรรมการซื้อทั้งหมด
- ความหมายทางธุรกิจ: ฟีเจอร์ที่ขึ้นอันดับบนคือ `Recency`, `Customer_Tenure_Days` และ `MntWines`, `MntFruits`, `MntMeatProducts`, `MntFishProducts`, `MntSweetProducts`, `MntGoldProds` แต่การจัดอันดับความสำคัญไม่บอกทิศทางของผล จึงยังไม่สรุปว่าฟีเจอร์เหล่านี้เพิ่มหรือลดโอกาสตอบรับ

5.6 อภิปราย

- **regression:** ผลดีที่สุดมาจาก GBDT (R^2 0.82 เทียบ 0.72–0.78 ของ neural network) และผลซ้ำใน 5-fold (นำ MLP หลาย Layer ทุก fold และนำ NAM 4

จาก 5 fold) สาเหตุที่เป็นไปได้คือต้นไม้จัดการตารางขนาดเล็กได้โดยไม่ต้องปรับสเกล แต่รายงานนี้ยังไม่ได้ทดสอบสมมติฐานนี้โดยตรง ผลนี้สอดคล้องกับบทวิจัยที่พบว่าต้นไม้ยังคงเหนือกว่า neural network บนข้อมูลตารางทั่วไป [13]

- **classify:** สามกลุ่ม architecture ใกล้กันเมื่อดู 5-fold (F1 0.55–0.59) และ ROC-AUC ของ LightGBM (0.890) สูงที่สุดในทุกโมเดลบน split เดียว แต่ผลต่างในระดับนี้ยังแยกจากความแปรผันไม่ได้
- ความซับซ้อนไม่ได้ช่วยเสมอ**:** โมเดลที่ใหม่กว่า (FT-Transformer-lite, NAM) ไม่ได้ชนะ LightGBM ในงานใด และ NAM แพ้ MLP หลาย Layer ใน regression
- ความอธิบายได้มีราคา**:** NAM ให้กราฟรูปร่างต่อฟีเจอร์ได้ แลกกับ R^2 ที่ต่ำกว่า MLP หลาย Layer ราว 0.016 (5-fold)

5.7 ข้อจำกัดของการทดลอง

- ไม่มี validation set แยก: V1.1, V1.2, V2.0 และ V2.1 เลือก checkpoint บนชุดทดสอบหรือ fold ที่ใช้รายงาน ผลจึงเอียงบวกเล็กน้อย (V3.0/V3.1 early stopping บน 10% ของ train จึงไม่มีปัญหานี้)
- ชุดทดสอบ classify มีลูกค้าที่ตอบรับแค่ 67 แถว แต่ละแถวในกลุ่มนี้เปลี่ยน recall ได้ราว 1.5 จุดเปอร์เซ็นต์ จึงต้องพึ่ง 5-fold
- 5-fold ทำซ้ำแค่ seed เดียว (42) จึงไม่รู้ว่า SD จากการแบ่ง fold ต่างจาก SD จาก seed ในการเริ่มน้ำหนักอย่างไร
- การวิเคราะห์ EDA (หัวข้อ 2.3–2.8) คำนวณก่อนลบแถว Income $666,666 จึงใช้ 2,237 แถว (classify) และ 2,213 แถว (regression) ไม่ใช่ 2,236 และ 2,212 แถวของการเทรน
- ไม่ได้รัน baseline ง่าย (logistic regression, linear regression) และไม่มีภาพ confusion matrix หรือเส้น ROC ในงานนี้
- ทุกเวอร์ชันใช้ threshold 0.5 คงที่ ไม่มีการปรับจุดตัดให้เหมาะกับ F1
- Scratch GBDT สร้างต้นไม้แบบ depth-wise ในขณะที่ LightGBM เป็น leaf-wise จึงไม่ใช่สำเนาเหมือนทุกจุด

บทที่ **6** สรุป ข้อจำกัด และแนวทางต่อยอด

6.1 สรุปผล

- ข้อสรุปหลัก**:** Income สัมพันธ์แรงกับฟีเจอร์ที่มีอยู่ (เช่น MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds และ NumCatalogPurchases) โมเดลจึงทำนายได้ดี (R^2 0.72–0.82) ส่วน Response ฟีเจอร์ที่มีให้สัญญาณไม่พอ ทุกโมเดลจึงได้ F1 ราว 0.55–0.60 การทำนายให้ดีขึ้นน่าจะต้องใช้ฟีเจอร์หรือข้อมูลเพิ่ม
- **Model** ที่เลือก**: LightGBM** ทั้งสอง regression ได้ R^2 0.8260 ± 0.0427 บน 5-fold สูงกว่า neural network ทุกแบบ ส่วน classify ได้ F1 ราว 0.60 ใกล้เคียง FT-Transformer-lite แต่ได้ AUC สูงสุด (0.890) ฝึกเร็วและไม่ต้องปรับสเกลฟีเจอร์
- **regression (Income):** LightGBM ดีที่สุด R^2 0.8212 บน split เดียว และ 0.8260 ± 0.0427 บน 5-fold (RMSE $9,217) ส่วน neural network ทุกแบบอยู่ที่ R^2 0.72–0.78
- **classify (Response):** FT-Transformer-lite และ LightGBM ได้ F1 ราว 0.60 บน split เดียว และ 0.58–0.59 บน 5-fold สูงกว่า MLP หลาย Layer (0.55) แต่ต่างกันเองไม่เกิน 0.010 ซึ่งเล็กกว่า SD ของ fold มาก
- เส้นทางการพัฒนา**:** ขยายโมเดลเป็น MLP หลาย Layer (V1.1) ได้ผลทั้งสองงาน การตัดฟีเจอร์ 2/3 (V1.2) ทำให้แย่ลง architecture ใหม่ (V2.0) ช่วย classify แต่ทำให้ regression แย่ลง และ GBDT (V3.0) ได้ผลดีที่สุดโดยรวม
- การคำนวณจากศูนย์ด้วย **NumPy:** ตรงกับไลบรารีอ้างอิงใน 8 จาก 10 กรณีของ split เดียว และใน 5-fold ทุกกรณี (ส่วนต่าง F1 เฉลี่ย ≤ 0.018, R^2 ≤ 0.005) อีก 2 กรณีที่ไม่ผ่านเกิดจากการเลือกจุดหยุดฝึก ไม่ใช่สูตรคณิตศาสตร์ (หัวข้อ 5.4)

6.2 ข้อเสนอการใช้งาน

- ถ้าต้องการทำนายอย่างเดียว: ใช้ LightGBM ทั้งสองงาน เพราะดีที่สุดหรือเสมอกับดีที่สุด ฝึกเร็ว และไม่ต้องปรับสเกลฟีเจอร์
- ถ้าต้องการอธิบายผลต่อฟีเจอร์: NAM (regression) ให้กราฟรูปร่างต่อฟีเจอร์แลกกับ R^2 ที่ต่ำกว่า LightGBM ราว 0.087 บน 5-fold (0.7395 เทียบ 0.8260)

- classify: ตัวเลข F1 ราว 0.6 ยังเป็นระดับที่ควรใช้จัดอันดับลูกค้า ไม่ควรใช้ตัดสินรายบุคคลโดยตรง และอย่าเปรียบเทียบเกณฑ์ threshold 0.5 โดยไม่ปรับให้เข้ากับต้นทุนของแคมเปญ

6.3 ข้อจำกัด

ข้อจำกัดทั้งหมดอยู่ในหัวข้อ 5.7 ข้อที่กระทบข้อสรุปมากที่สุดคือ ไม่มี validation set แยก (checkpoint ของโมเดล neural network เลือกบนข้อมูลที่ใช้รายงานผล) และชุดทดสอบ classify มีลูกค้าที่ตอบรับเพียง 67 แถว

6.4 แนวทางต่อยอด

- แยก validation set จริง หรือใช้ nested cross-validation เพื่อให้ตัวเลขรายงานปราศจากการเลือก checkpoint บนชุดเดียวกัน
- ทำซ้ำ 5-fold ด้วยหลาย seed เพื่อแยก SD จากการแบ่ง fold ออกจาก SD จากการสุ่มน้ำหนักเริ่มต้น
- แก้เกณฑ์ early stopping ของ Scratch GBDT (เช่น ใช้ F1 บน validation หรือกำหนดจำนวนรอบคงที่) เพื่อไม่ให้เกิดการหยุดที่รอบ 8 อีก
- ปรับ threshold ของ classify ตามต้นทุนแคมเปญ และรายงาน confusion matrix กับเส้น ROC / precision-recall
- เพิ่ม baseline ง่าย ๆ (logistic regression, linear regression) เพื่อวัดว่า neural network และ GBDT เพิ่มค่าเหนือโมเดลเชิงเส้นเท่าไร
- รัน EDA ซ้ำบนแถวหลังตัดค่าผิดปกติ (2,236 แถวใน classify และ 2,212 แถวใน regression) ให้ตรงกับข้อมูลที่ใช้เทรน